Engineering Science and Technology, an International Journal 47 (2023) 101525

Contents lists available at ScienceDirect

Engineering Science and Technology, an International Journal

journal homepage: www.elsevier.com/locate/jestch

Genetic based density peak possibilistic fuzzy *c*-means algorithms to cluster analysis- a case study on customer segmentation

R.J. Kuo [a], Muhammad Naufal Alfareza [a], Thi Phuong Quyen Nguyen [b,*]

[a] Department of Industrial Management, National Taiwan University of Science and Technology, No. 43, Section 4, Kee-Lung Road, Taipei 106, Taiwan
[b] Faculty of Project Management, The University of Danang-University of Science and Technology, 54 Nguyen Luong Bang, Danang, Vietnam

ARTICLE INFO

Keywords:
Customer segmentation
Density peak clustering
Genetic algorithm
Possibilistic fuzzy c-means algorithm

ABSTRACT

Finding the target consumers for a business depends heavily on its market segmentation approach. Applying clustering analysis to consumer segmentation is one of the most common methods. However, most clustering algorithms can easily fall into the local optimum solution. Besides, it can be challenging to handle noise and outliers and determine the optimal parameters. The initial cluster centers can also affect the clustering result. Thus, this study proposes a clustering algorithm that first employs density peak clustering to obtain the initial cluster centers. Then, the proposed method integrates genetic algorithm (GA) with possibilistic fuzzy c-means (PFCM) algorithm, where GA is used to optimize the cluster centers and the parameters of the PFCM algorithm to overcome the problems above. Using eleven benchmark datasets, the computational results demonstrate that the proposed algorithm can provide better and more robust results in terms of accuracy, adjusted rand index (ARI), and normalized mutual information (NMI) compared to previous clustering algorithms. Additionally, the proposed algorithm is used to segment customers of a retail company in a dataset containing recency, frequency, and monetary (RFM) variables. The clustering result for customer segmentation is also very promising.

1. Introduction

To meet diverse customer needs, market segmentation has become an important issue for a company to pay attention to. It can help a company recognize more about the preferences of its customers and implement different marketing strategies for targeted segments to improve customer satisfaction and increase the company revenue [1]. However, poor segmentation due to the wrong choice of methodology can fail to identify groups worth pursuing or retaining and potentially lead to a huge waste of resources due to the firm tailoring its market [2].

One of the most common techniques to segment the market is to use clustering analysis. The clustering method is an efficient way to do market segmentation because it can distinctly group clusters [3]. Many clustering algorithms have been proposed with their respective advantages and disadvantages. The most challenging task in the clustering algorithm is the sensitivity to initial cluster centers because they have a great impact on the clustering result [4]. In addition, there are other issues for clustering analysis, such as the ability to handle data containing outliers and noise as well as a local optimum solution [5]. Nonetheless, there is no single clustering technique that is capable of solving all the mentioned problems and providing a good result for all different types of data.

Several clustering methods have been published in which density-based clustering approach gives more advantages than other methods for the capabilities of noise handling. Wang et al. [6] proposed a variation of density peak clustering (DPC) which implemented a clustering tasks in terms of multiple density distribution. Li et al. [7] combined DPC with a fuzzy semantic model to provide a new local density metric as well as a semantic distance for better cluster center selection. Liu et al. developed a density peak-based fuzzy *c*-means (DP-FCM) algorithm to solve the sensitivity of the initial cluster centers and improve the clustering accuracy [8]. However, one of the drawbacks of DP-FCM algorithm is that cluster centers are selected manually from the decision graph, which cannot be determined easily. Besides, FCM algorithm has weak performance for noisy data [9]. This study aims to provide an enhancement of the DP-FCM algorithm. First, possibilistic fuzzy *c*-means algorithm (PFCM) [10], which is a combination of FCM algorithm and possibilistic *c*-means (PCM) [11] algorithm, is used to replace FCM in the DP-FCM algorithm. The PFCM algorithm included the possibilistic terms and the fuzzy terms into its objective function to tackle noise and

* Corresponding author.
E-mail addresses: rjkuo@mail.ntust.edu.tw (R.J. Kuo), naufal.alfareza@gmail.com (M.N. Alfareza), ntpquyen@dut.udn.vn (T.P.Q. Nguyen).

https://doi.org/10.1016/j.jestch.2023.101525
Received 7 February 2023; Received in revised form 25 July 2023; Accepted 4 September 2023
Available online 17 September 2023
2215-0986/© 2023 THE AUTHORS. Published by Elsevier BV on behalf of Karabuk University. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

outliers data. Besides, the PFCM algorithm can omit the step of scaling the typicality values, making it more effective for large datasets [10]. To deal with the issue of local optimal solutions, some studies combined clustering with metaheuristics to improve the clustering performance, such as genetic algorithm (GA), ant colony algorithms, and particle swarm organization [12–13]. Numerous studies have utilized GA to address the clustering problem or combined GA with clustering approaches in various ways, demonstrating that this strategy can avoid falling to the local optimal solution to enhance the performance of clustering [14]. According to Chehouri et al. [15], utilizing GA can generate better results and a more consistent outcome compared to using other *meta*-heuristics. Moreover, GA may converge to a global optimal solution [16]. Therefore, GA is selected to aim at providing the global optimal solution. Thus, this study proposes an integration among DPC, PFCM, and GA, which is named DP-GA-PFCM algorithm. DPC clustering is employed to find the initial cluster centers. Thereafter, GA is embedded in the clustering process to find the optimal parameters and cluster centers for PFCM algorithm simultaneously. The optimal results from GA are then implemented on the PFCM algorithm to obtain the final clustering result. The proposed DP-GA-PFCM is expected to provide an outstanding result due to several novelties and contributions:

(1) This is a combination of density-based and fuzzy-based clustering approaches.
(2) The proposed method can deal with noise and outlier data.
(3) The optimal parameters and cluster centers can be simultaneously exploited.
(4) The clustering performance is measured by multiple indicators such as accuracy, adjusted rand index (ARI), and normalized mutual information (NMI).

The proposed DP-GA-PFCM algorithm is implemented on multiple datasets UCI Machine Learning Repository and the results are compared with other clustering methods in terms of accuracy, ARI, and NMI. Furthermore, this study also applies the proposed algorithm to conduct market segmentation for a case study in a retail business.

The remaining sections of this study are structured as follows. Section 2 gives some general backgrounds of the related literature. The proposed algorithm is presented in Section 3. Section 4 provides the results of the model evaluation. A case study outcome is displayed in Section 5. Finally, the concluding remarks are made in Section 6.

2. I literature review

This section provides a literature survey related to the current study including the density peak fuzzy *c*-means algorithm, possibilistic fuzzy *c*-means algorithm, and genetic algorithm.

2.1. Density peak fuzzy c-means algorithm

Density peak fuzzy *c*-means algorithm is an integration of DPC and FCM algorithms. This algorithm calculates two values which are local density ρ_i of each data point *i*, and its distance δ_i from point *i* of higher density. Also, it has an assumption that cluster centers are surrounded by neighbors with lower local density and they are at a relatively large distance from any points with a higher local density [17]. The procedure of DPFCM algorithm is as follows:

Step 1: Calculate the local density. Each record in the dataset is calculated its local density with Eq. (1) for discrete value and Eq. (2) for continuous value. The local density of data point *i* for $i = \{1, 2,\ldots, n\}$ for discrete value is defined as:

$$\rho_i = \sum_j X^{(dis_{ij} - d_c)} \tag{1}$$

where $X^{(x)} \begin{cases} 1, x < 0 \\ 0, x \geq 0 \end{cases}$. For continuous value, a Gaussian kernel function is used as follows:

$$\rho_i = \sum_{j \neq i} e^{-\left(\frac{dis_{ij}}{d_c}\right)^2} \tag{2}$$

where d_c is a cutoff distance, and dis_{ij} is the Euclidean distance between data *i* and *j* for *i, j* = {1, 2,…, *n*}.

Step 2: Calculate the distance. The distance is calculated from each data point *i* for $i = \{1, 2,\ldots, n\}$ with Eq. (3). For the data point with the highest local density, the distance is $\max_j(dis_{ij})$.

$$\delta_i = \begin{cases} \min_{j:\rho_j > \rho_i}(dis_{ij}) \\ \max_j(dis_{ij}) \end{cases} \tag{3}$$

Step 3: Create a decision graph. DPC uses the local density as the x-axis and the distance as the y-axis to build a decision graph. From the decision graph, a data point that has large values both in local density and distance will be the cluster center.

Step 4: Initialize the membership matrix. The membership matrix is defined by generating random numbers with the sum of each row equal to 1. The output of the membership matrix can be illustrated in Eq. (4).

$$u_{ik} = \begin{bmatrix} u_{11} & \cdots & u_{1m} \\ \vdots & \ddots & \vdots \\ u_{n1} & \cdots & u_{nm} \end{bmatrix}; 1 \leq i \leq n; 1 \leq k \leq c, \tag{4}$$

u_{ik} represents the membership between the cluster centers and the data point where n is the total data and $j = 1,2,\ldots,m$ is the number of attributes. Each data point is assigned a degree of membership for each cluster.

Step 5: Calculate the centroids. After defining the membership matrix, then calculate the cluster centers with Eq. (5).

$$V_k = \frac{\sum_{i=1}^{n}(u_{ik})^w X_{ij}}{\sum_{i=1}^{n}(u_{ik})^w}; 1 \leq i \leq n; 1 \leq k \leq c, \tag{5}$$

From Eq. (5), the cluster centers obtained by the summation of the multiplication between the data point with the membership powered by the rank weight *w* where X_{ij} means data point *i* with attribute *j* and $w > 1$. Then, the value is divided by the sum of the membership powered by *w*.

Step 6: Calculate the objective function. The minimum objective function of FCM algorithm as shown in Eq. (6) can be obtained iteratively as follows:

$$P_t = \sum_{i=1}^{n}\sum_{k=i}^{c}\left(\left[\sum_{j=1}^{m}\left(X_{ij} - V_{kj}\right)^2\right](u_{ik})^w\right) \tag{6}$$

Step 7: Update the membership. When the value of the objective function has not met the accepted threshold or the number of iterations has not reached the specified maximum number of iterations, an updated membership can be carried out using Eq. (7). Then, calculate again the cluster centers as shown in Step 5 and so on.

$$u_{ik} = \frac{\left[\sum_{j=1}^{m}\left(X_{ij} - V_{kj}\right)^2\right]^{\frac{-1}{w-1}}}{\sum_{i=1}^{c}\left[\sum_{j=1}^{m}\left(X_{ij} - V_{kj}\right)^2\right]^{\frac{-1}{w-1}}} \tag{7}$$

2.2. Possibilistic fuzzy c-means algorithm

Possibilistic fuzzy c-means (PFCM) algorithm is an algorithm proposed by Pal et al. [10]. This algorithm is the combination between FCM and possibilistic c-means (PCM) algorithms. PFCM algorithm has two kinds of memberships, possibilistic membership which measures the absolute degree of typicality of a point in any particular cluster, and fuzzy membership which measures the relative degree of sharing points

along the clusters [18]. The procedure for PFCM algorithm is as follows:

Step 1: Initialize the partition matrix using Eq. (4).

Step 2: Calculate the typicality matrix. Besides the membership degree or partition matrix, a typicality matrix is required in the calculation of the PFCM algorithm. The typicality matrix is a typicality value to determine which data point belongs to which cluster. The typicality matrix is calculated using Eq. (8) as follows:

$$t_{ik} = \left[1 + \left(\frac{b\left[\sum_{j=1}^{m}(X_{ij} - V_{kj})^2\right]}{\gamma_k}\right)^{\frac{1}{\eta-1}}\right]^{-1} \tag{8}$$

R03 where γ_k is calculated as follows:

$$\gamma_k = K\frac{\sum_{i=1}^{n}\left((\mu_{ik})^w\left[\sum_{j=1}^{m}(X_{ij} - V_{kj})^2\right]\right)}{\sum_{i=1}^{n}(\mu_{ik})^w}, \tag{9}$$

Step 3: Calculate the cluster centers. After defining the partition matrix and the typicality matrix, then calculate the cluster centers as follows:

$$V_k = \frac{\sum_{i=1}^{n}(a{\mu_{ik}}^w + b{t_{ik}}^\eta)X_{ij}}{\sum_{i=1}^{n}(a{\mu_{ik}}^w + b{t_{ik}}^\eta)} \tag{10}$$

Step 4: Calculate the objective function. The objective function of PFCM is shown in Eq. (11).

$$P_t = \sum_{i=1}^{n}\sum_{k=1}^{c}(a{\mu_{ik}}^w + b{t_{ik}}^\eta)\left[\sum_{j=1}^{m}(X_{ij} - V_{kj})^2\right] + \sum_{k=1}^{c}\gamma_k\sum_{i=1}^{n}(1 - t_{ik})^\eta \tag{11}$$

The constants 'a' and 'b' are defined as the relative importance of fuzzy membership and typicality values in the objective function, respectively.

Step 5: Update the partition matrix and typicality matrix. When the value of the objective function has not met the accepted threshold or the number of iterations has not reached the specified maximum number of iterations, an updated partition matrix can be carried out using Eq. (12) and an updated typicality matrix can be carried out using Eq. (8). Then, calculate again the cluster centers as shown in Step 3 and so on.

R12 $$\mu_{ik} = \left[\sum_{j=1}^{m}(X_{ij} - V_{kj})^2\right]^{\frac{-1}{w-1}} / \sum_{i=1}^{c}\left[\sum_{j=1}^{m}(X_{ij} - V_{kj})^2\right]^{\frac{-1}{w-1}} \tag{12}$$

2.3. Genetic algorithm

One of the most well-known metaheuristic algorithms is the genetic algorithm (GA), which is based on the theory of natural evolution [19]. In GA, the fitness value for each individual affects the possibility of reproduction. The better chromosomes they have, the more likely they are to be selected for the next generation [19–20]. Haldurai et al. [20] described the detailed process for GA as follows:

Step 1: Initialization: Representation of a problem is an important concept in solving using GA. A population consists of a collection of chromosomes under the problem to be solved.

Step 2: Evaluation: This step is to measure the fitness function of each chromosome in the population. The fitness function is a measure of an individual's performance which is identical to the objective function of the optimized problem. The objective function can be minimization or maximization. The higher or the lower the fitness value, the higher the probability the individual will survive.

Step 3: Selection: It is carried out to select chromosomes in the population that has the best fitness value. Chromosomes that meet the predetermined fitness criteria are more likely to be selected for the next generation. Several commonly used selection methods include roulette wheel selection, rank-based fitness assignment, tournament selection, and elitism.

Step 4: Crossover: This is the process of combining several values in an individual to produce a new individual. It begins with selecting a pair of individuals from the population, then if it meets the specified crossover rate, a crossover will be carried out. Several methods used in crossovers are single-point crossover, two-point crossover, and uniform-point crossover.

Step 5: Mutation: This is the process of changing an individual's value at a certain location. The mutation process is carried out by replacing one randomly selected value in a chromosome with a new random value. Otherwise, in a binary number problem, individuals who meet the mutation rate will be changed from 0 to 1 or vice versa.

Step 6: Back to step 2. The algorithm will stop when it has reached the specified number of generations or has reached an acceptable fitness value.

3. Methodology

A novel clustering technique, that combines DPC, GA, and PFCM algorithms (denoted as DP-GA-PFCM) is proposed to improve the clustering performance. The proposed DP-GA-PFCM algorithm also can solve the sensitivity of initial cluster centers as well as can deal with noise and outlier data. As mentioned earlier, the PFCM algorithm can tackle noise and outliers based on its typicality term. The proposed method uses the concept of density peak to first obtain the cluster centers. Then the revealed cluster centers are used as the initialization of the PFCM algorithm. However, the PFCM algorithm's parameters are not easy to determine, and cluster centers derived from density peak results may not always give the most accurate clustering result. Therefore, GA is employed to find the optimal parameters and cluster centers for the PFCM algorithm. The following subsection will explain in more detail the proposed method DP-GA-PFCM.

3.1. Data preprocessing

R13 A MinMax scaler is used to preprocess the dataset before analysis. This scale changes the value of each attribute to a range between 0 and 1. The scaled value, X_{scaled}, can be obtained as follows:

$$X_{scaled} = \frac{X_i - min(x)}{max(x) - min(x)} \tag{13}$$

where X_i represents the original value, min(x) represents the minimum value of all the attribute data, and max(x) represents the maximal value of all the attribute data.

3.2. Proposed DP-GA-PFCM algorithm

Fig. 1 shows the algorithm framework of the proposed DP-GA-PFCM algorithm, which consists of three stages. In the first stage, the DPC is employed to identify initial cluster centers based on a decision graph, where the decision graph can also identify the presence of outliers. Cluster centers consist of data points that have the highest density. However, the obtained cluster centers may not produce satisfactory results on some datasets because it allows the existence of outliers that will affect the clustering results. To overcome the problem of outliers, the PFCM algorithm is applied in which the cluster centers obtained from DPC are used for the initialization of the PFCM algorithm. Since it's not easy to determine the optimal parameters for PFCM, we employ a GA to obtain the optimal cluster centers and four required parameters in the PFCM algorithm, i.e., a, b, w, η. The PFCM algorithm is embedded inside the GA and its purpose is only to calculate the fitness of chromosomes. Therefore, PFCM algorithm is only run one iteration for each GA generation. Then, in the third stage, the PFCM algorithm is implemented iteratively to fine-tune the result from the GA. In summary, after getting the initial cluster centers from the density peak clustering, which are then optimized using a GA involving a PFCM algorithm operation for calculating the fitness, the PFCM algorithm is performed to obtain the

R05

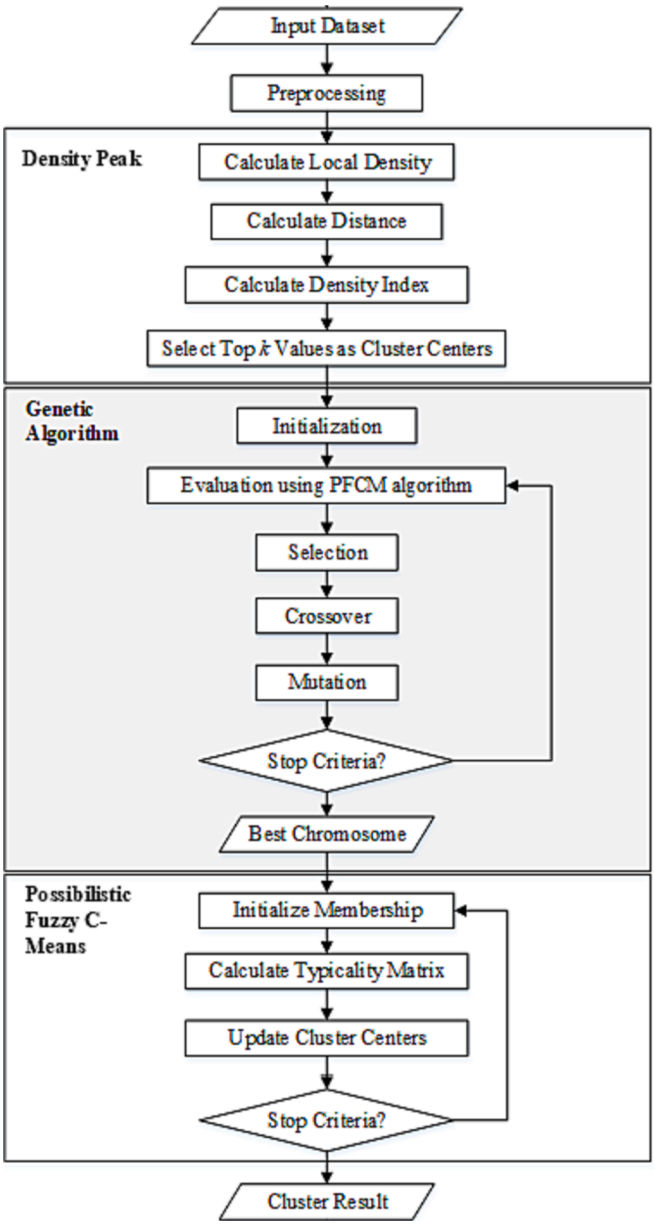


Fig. 1. A framework of the DP-GA-PFCM algorithm.

final clustering results. Note that the implementation of PFCM in the third stage is similar to its original version in the way it updates the fuzzy membership and the typicality from the optimal cluster centers obtained from GA's result. Especially, the typicality is now automatically scaled up by the γ_k in Eq. (9).

The cluster centers used for GA's starting population are derived from the density peak clustering results. Fig. 2 illustrates the chromosome's representation. The number of cluster centers depends on the number of clusters, n, and the number of attributes, m, in the data used. In addition, a, b, w, and eta are the four required parameters in the PFCM algorithm where these values are greater than zero.

The procedures of the proposed DP-GA-PFCM algorithm are listed as follows:

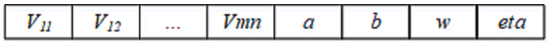


Fig. 2. Representation of chromosome.

Step 1: Input and preprocess the dataset using Eq. (13).

Step 2: Calculate local density and the distance using Eq. 2–3, respectively.

Step 3: Calculate the distance index by multiplying the local density and the distance.

Step 4: Select top k values as initial cluster centers.

Step 5: Initialize the population according to the number of chromosomes.

Step 6: Evaluate the chromosomes in the population to minimize the within-cluster sum of squared errors using PFCM algorithm (Eq. (6).

Step 7: Implement selection using the roulette wheel selection mechanism.

Step 8: Implement crossover and mutation process.

The chromosome consists of two parts, i.e., PFCM's parameters and cluster centers, which are all represented by real value. Traditional one-point crossover or multi-point crossover is not suitable since the exchange genes need to fit each particular part of the chromosome. Thus, this study uses the crossover function of real-coded GA [21] to generate the new chromosome, which is represented as follows:

$$O_1 = rP_1 + (1-r)P_2, \tag{14}$$

$$O_2 = rP_2 + (1-r)P_1, \tag{15}$$

where O_1 and O_2 are two offspring chromosomes generated from two parent chromosomes P_1 and P_2, r is a random number from 0 and 1.

Similarly, the mutation process is performed based on the following function [21]:

$$O_i = P_i + s_i \times r_i \times q_i \tag{16}$$

where $q_i = 2^{-uk}$, s_i and u are two random numbers in which $s_i \in \{-1, +1\}$ and $u \in [0, 1]$, r_i is the mutation range which is set at 10% as standard, and k is mutation precision ($k \in \{4, 5, \cdots 20\}$).

Step 9: Repeat steps 6–8 and stop if the maximum number of iterations is satisfied to obtain the optimal cluster centers and parameters for PFCM.

Step 10: Initialize or update membership using Eq. (7) based on cluster centers obtained from Step 9.

Step 11: Calculate the typicality matrix using Eq. (8).

Step 12: Update cluster centers using Eq. (10).

Step 13: Repeat Steps 10–12 and stop if the maximum number of iterations is reached or when the difference in the objective function has met the accepted threshold.

The Pseudocode of the proposed algorithm is defined as follows:

Algorithm 1. DP-GA-PFCM Algorithm

Input: k (number of clusters), *MAX_ITER* (maximum of iterations)
Output: L (set of k cluster label)

Step
Calculate local density
Calculate the distance
Calculate the distance index
Select top k values
Initialize population
while $i <$ *MAX_ITER*, **do**
Calculate fitness using PFCM algorithm
Selection
Crossover
Mutation
end while Select the best chromosome
Initialize membership
while $j <$ *MAX_ITER*, **do**
Calculate typicality matrix
Update cluster centers
Update membership
end while
Report L as a set of cluster labels

3.3. Performance measurement

Performance measurement is performed to validate whether the proposed algorithm gives better results. There are three measurements used which are accuracy, adjusted rand index (ARI), and normalized mutual information (NMI). To quantify the accuracy, the correct amount of data in each cluster is divided by the total amount of data. Accuracy can be defined as:

R18

$$Accuracy = \frac{\sum_{i=1}^{k} \max(a_i)}{n} \quad (17)$$

where max(a_i) represents the number of data points correctly predicted both in i^{th} cluster and its corresponding true class, and n is the total number of data points.

ARI measures the similarity between groups by considering all sample pairs and counting the pairs assigned to the same or different clusters. ARI calculation can be defined as:

$$ARI = \frac{RI - ExpectedRI}{\max(RI) - ExpectedRI}, \quad (18)$$

where *RI* represents the similarity between two clusters by considering all pairs of samples, *Expected RI* represents the average value of each cluster *RI,* and max*(RI)* represents the maximum value of each cluster RI.

NMI is another measurement that is often used to measure the performance of clustering algorithms. NMI measures how much information a variable can get from other variables that have been normalized. The NMI value is in the interval of zero and 1. If it is 0, then it means that there is no mutual information or that the two variables are independent of each other. If the value is 1, then the two variables are mutually dependent. NMI calculation can be defined as:

R18

$$NMI = I(X,Y)/\sqrt{H(X)H(Y)} \quad (19)$$

where *I(X, Y)* represents the amount of mutual information, and *H (X)* and *H(Y)* refers to the entropy of information for the variable X and Y.

4. Experimental results

R24 This section presents the experimental results to evaluate the proposed algorithm's performance. All algorithms are coded in Python and run on a Windows 10 computer with an Intel Core i7 processor and 20 GB RAM.

4.1. Datasets

R27 To evaluate the performance of the proposed algorithm, this study used 14 benchmark datasets from the UCI machine learning repository R14 (https://archive.ics.uci.edu/ml/). Table 1 shows the description of each dataset used, where *K* represents the number of clusters that are predetermined, *F* represents the number of attributes, and *N* represents the number of records. Since the proposed algorithm focuses on overcoming the presence of noise and dataset outliers, some datasets containing outliers are Flame, Wine, Glass, Ecoli, Compound, Seed, Breast Tissue, and Pima.

Table 1
Description of the datasets.

No.	Dataset	*K*	*F*	*N*
1.	Iris	3	4	150
2.	Flame	2	2	240
3.	Jain	2	2	373
4.	Wine	3	13	178
R15 5.	Glass	7	9	214
6.	Ecoli	8	7	336
7.	Compound	6	2	399
8.	Seed	3	7	210
9.	Breast Tissue	6	9	106
10.	Aggregation	7	2	788
11.	Pima	2	8	768
R14 12	Room Occupancy	2	7	20,560
13	Rice	2	7	3810
14	Wifi	4	7	2000

4.2. Parameters setup

In essence, the proposed algorithm is the result of incorporating three methods: DPC, PFCM, and metaheuristics. However, each method requires its parameters. A cutoff distance of 2% is set for DPC. PFCM R11 algorithm also has its parameters. As these parameters are difficult to determine, a metaheuristic is used to find the most efficient combination of these parameters. Nevertheless, the metaheuristic requires some parameters to be determined. In this study, GA is the main *meta*-heuristic approach that is used to combine with DPC to find the optimal cluster centers and optimal parameters for the PFCM algorithm. Besides, some other *meta*-heuristics such as particle swarm optimization (PSO) [22], sine–cosine algorithm (SCA) [23], and Forensic-Based Investigation (FBI) [24] are also employed to evaluate the effectiveness of the *meta*-heuristic method. Due to the comparison purpose, the population size and the maximum number of iterations are the same for all metaheuristics. Maximum iterations are being used as the criterion for stopping.

In metaheuristics, there is no single best value to represent the parameters used. A general factorial design was used to test several parameters at distinct levels to discover the optimal combination. For GA, two different values for the crossover rate and the mutation rate are selected in a range of 0 to 1. Generally, the crossover rate is high while the mutation rate is low. Hassanat et al. [20] pointed out that the crossover rates and the mutation rate are two critical factors in the search for GA. Besides, population size is the most important parameter in influencing choosing the crossover rate and mutation rate. While mutation rates were the most reliable at small population sizes, crossover rates varied based on population size. However, moderate population size would help the genetic operation, which involves mechanisms for selection, crossover, and mutation, operate well. Thus, to reduce the number of factors that can lead to reducing the complexity of the factorial design, the population size is not considered in the experiment. The population size and the number of iterations are set as 80 and 100, respectively, after several trials for all the algorithms.

Likewise, PSO uses different inertia weights and constants. As a result, GA and PSO each execute four experiments with different combinations of parameters 30 times. The levels of each factor in the experiment are shown in Table 2. Based on previous investigations that used GA and PSO in clustering [25–27], the crossover rate is 0.7 and 0.8, whereas the mutation rate is 0.01 and 0.03. For PSO, two constants, c1, and c2, are set equally at two levels, i.e., 1 and 2, while two levels of the inertia weight begin with 0.4 and 0.5 and are gradually increased to 0.9 and 0.8, respectively. The clustering accuracy is used to evaluate the performance of each combination. The results for the average accuracy for GA and PSO respectively can be seen in Table 3 and Table 4. Then, a combination of parameters is taken that produces the highest average value from 14 benchmark datasets which are shown in Table 5.

Table 2
Tuning parameters.

	Level 1	Level 2
Crossover rate	0.7	0.8 R08
Mutation rate	0.01	0.03
Inertia weight	[0.4 0.9]	[0.5 0.8]
Constants ($c_1 = c_2$)	1	2

Table 3
The average accuracy of GA on different tuning parameters.

Crossover rate	0.7		0.8	
Mutation rate	0.01	0.03	0.01	0.03
Iris	0.919	0.908	0.914	0.912
Flame	0.846	0.848	0.848	0.845
Jain	0.878	0.871	0.882	0.876
Wine	0.943	0.948	0.955	0.949
Glass	0.575	0.569	0.581	0.565
Ecoli	0.754	0.750	0.777	0.763
Seed	0.896	0.898	0.900	0.894
Aggregation	0.910	0.906	0.910	0.909
Compound	0.814	0.836	0.845	0.834
Breast Tissue	0.611	0.608	0.615	0.615
Pima	0.668	0.669	0.710	0.677
Room Occupancy	0.862	0.874	0.896	0.885
Rice	0.896	0.890	0.924	0.918
Wifi	0.931	0.926	0.963	0.943

Table 4
The average accuracy of PSO on different tuning parameters.

Initial weight	[0.5, 0.8]		[0.4, 0.9]	
Constants ($c_1 = c_2$)	1	2	1	2
Iris	0.892	0.889	0.891	0.899
Flame	0.843	0.843	0.842	0.843
Jain	0.882	0.881	0.881	0.881
Wine	0.950	0.951	0.950	0.951
Glass	0.571	0.558	0.554	0.561
Ecoli	0.789	0.782	0.783	0.792
Seed	0.890	0.890	0.891	0.890
Aggregation	0.904	0.904	0.904	0.903
Compound	0.840	0.840	0.839	0.844
Breast Tissue	0.574	0.564	0.568	0.574
Pima	0.664	0.662	0.663	0.662
Room Occupancy	0.865	0.861	0.869	0.873
Rice	0.901	0.896	0.913	0.920
Wifi	0.923	0.912	0.927	0.929

Table 5
Parameters setup.

	GA	PSO	SCA	FBI
Population size	80	80	80	80
Maximum iteration	100	100	100	100
Crossover rate	0.8	–	–	–
Mutation rate	0.1	–	–	–
Inertia weight	–	[0.4 0.9]	–	–
Constants (c1 = c2)	–	2	–	–

4.3. Experimental results

In this study, the experiment is divided into two stages. The first stage is to compare the five clustering algorithms without using the metaheuristic. This stage aims to determine whether the DP-PFCM algorithm is superior to K-means algorithm, FCM algorithm, PFCM algorithm, and DP-FCM algorithms or not. The second stage is to compare the proposed clustering algorithms with the others using the metaheuristic. This stage aims to determine which metaheuristic algorithm is superior to others among GA, PSO, SCA, and FBI algorithms to get the result better or more stable.

4.3.1. First-stage experimental results

Table 6, Table 7, and Table 8 show the first-stage experimental results in terms of accuracy, ARI, and NMI, respectively. These results are the average values of 30 executions. The high value of each performance essentially indicates better performance. The numbers in the column indicate the algorithm used, where (1) represents *K*-means, (2) represents FCM, (3) represents PFCM, (4) represents DP-FCM, and (5) represents DP-PFCM. Each performance measure for the dataset is ranked in which the highest score will be ranked first and the smallest score will be ranked last.

After being ranked based on the highest value, the values are added up in each column to determine which algorithm performs better. As can be seen from Table 6, Table 7, and Table 8, the sum of rank values of the algorithm (5) or DP-PFCM algorithm is the smallest compared to other algorithms in terms of accuracy, ARI, and NMI. This algorithm performs better on 9 out of 14 datasets, which are Iris, Flame, Wine, Glass, Seed, Breast Tissue, Pima, Room Occupancy, and Wifi. It can be concluded that DP-PFCM can provide better results on accuracy, ARI, and NMI. Besides, Fig. 3 depicts the sum of ranks of the proposed DP-PFCM and other benchmark algorithms on the 14 tested datasets in the first stage of the experiment for better visualization. The smallest sum of ranks shows the best performance.

To determine whether there is a significant difference among the algorithms, a statistical test is conducted based on the experimental results for the three performance measures. In this study, the significance level is set at 5%. The decision-making criteria on this test are the *p*-value or Asymp. Sig. (2-tailed) ≤ 0.05 then H_0 is rejected, otherwise H_1 is rejected. As an initial step in determining the appropriate test, the normality test is performed beforehand to determine whether the data is normally distributed or not. The majority of the normality test results showed that data is not normally distributed. Thus, the Kruskal-Wallis test is conducted which can evaluate group differences by estimating the difference in rankings between groups. This test is a nonparametric test which means distribution-free or the test doesn't need normality

Table 6
First-stage accuracy experimental result.

Dataset	Average Accuracy					Rank				
	(1)	(2)	(3)	(4)	(5)	(1)	(2)	(3)	(4)	(5)
Iris	0.886	0.892	0.899	0.892	0.908	5	3	2	4	1
Flame	0.843	0.848	0.853	0.851	0.855	5	4	2	3	1
Jain	0.883	0.884	0.878	0.883	0.879	3	1	5	2	4
Wine	0.944	0.944	0.945	0.941	0.952	4	3	2	5	1
Glass	0.546	0.556	0.564	0.558	0.566	5	4	2	3	1
Ecoli	0.813	0.823	0.822	0.831	0.817	5	2	3	1	4
Seed	0.891	0.899	0.895	0.898	0.899	5	2	4	3	1
Aggregation	0.911	0.898	0.905	0.908	0.909	1	5	4	3	2
Compound	0.813	0.830	0.853	0.832	0.838	5	4	1	3	2
Breast Tissue	0.556	0.600	0.600	0.557	0.611	5	3	2	4	1
Pima	0.665	0.668	0.672	0.669	0.681	5	4	2	3	1
Room Occupancy	0.810	0.815	0.852	0.857	0.862	5	4	3	2	1
Rice	0.905	0.905	0.907	0.919	0.912	4	4	3	1	2
Wifi	0.901	0.911	0.907	0.939	0.945	5	3	4	2	1
Sum of Rank						62	46	39	39	23

Table 7
First-stage ARI experimental result.

Dataset	Average ARI					Rank				
	(1)	(2)	(3)	(4)	(5)	(1)	(2)	(3)	(4)	(5)
Iris	0.714	0.727	0.742	0.725	0.759	5	3	2	4	1
Flame	0.469	0.481	0.497	0.492	0.501	5	4	2	3	1
Jain	0.579	0.584	0.564	0.581	0.569	3	1	5	2	4
Wine	0.836	0.837	0.839	0.827	0.858	4	3	2	5	1
Glass	0.169	0.159	0.183	0.173	0.194	4	5	2	3	1
Ecoli	0.457	0.413	0.433	0.395	0.419	1	4	2	5	3
Seed	0.706	0.724	0.719	0.723	0.729	5	2	4	3	1
Aggregation	0.720	0.684	0.692	0.708	0.683	1	4	3	2	5
Compound	0.524	0.531	0.564	0.531	0.547	5	3	1	3	2
Breast Tissue	0.277	0.301	0.327	0.336	0.393	5	4	3	2	1
Pima	0.087	0.108	0.112	0.109	0.115	5	4	2	3	1
Room Occupancy	0.731	0.698	0.712	0.748	0.739	3	5	4	1	2
Rice	0.831	0.830	0.792	0.864	0.853	3	4	5	1	2
Wifi	0.702	0.832	0.856	0.858	0.861	5	4	3	2	1
Sum of Rank						54	50	40	39	26

Table 8
First-stage NMI experimental result.

Dataset	Average NMI					Rank				
	(1)	(2)	(3)	(4)	(5)	(1)	(2)	(3)	(4)	(5)
Iris	0.740	0.741	0.750	0.742	0.764	5	4	2	3	1
Flame	0.423	0.438	0.447	0.446	0.455	5	4	2	3	1
Jain	0.522	0.543	0.535	0.545	0.533	5	2	3	1	4
Wine	0.822	0.826	0.825	0.813	0.842	4	2	3	5	1
Glass	0.331	0.312	0.343	0.329	0.353	3	5	2	4	1
Ecoli	0.604	0.587	0.609	0.580	0.596	2	4	1	5	3
Seed	0.680	0.689	0.684	0.688	0.701	5	2	4	3	1
Aggregation	0.843	0.814	0.834	0.830	0.836	1	5	3	4	2
Compound	0.697	0.699	0.719	0.701	0.708	5	4	1	3	2
Breast Tissue	0.514	0.529	0.543	0.519	0.562	4	3	2	5	1
Pima	0.044	0.063	0.071	0.066	0.070	5	4	1	3	2
Room Occupancy	0.802	0.647	0.626	0.798	0.843	2	4	5	3	1
Rice	0.865	0.852	0.860	0.927	0.876	3	5	4	1	2
Wifi	0.777	0.827	0.842	0.877	0.900	5	4	3	2	1
Sum of Rank						54	52	36	45	23

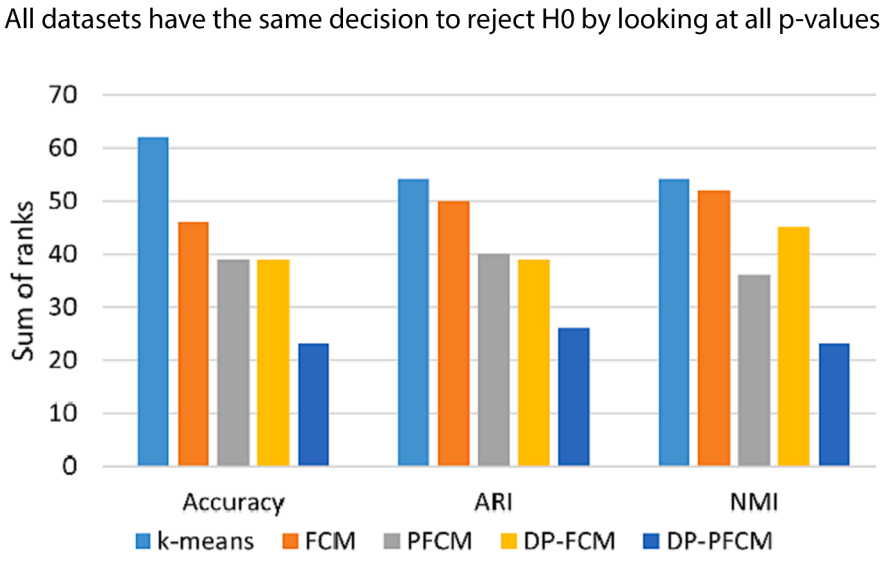


Fig. 3. The sum of ranks in the first stage of the experiment.

assumption. The null and alternative hypotheses for the Kruskal-Wallis test are as follows: H_0: There is no difference based on the algorithm used, and H_1: There is at least one different algorithm based on the algorithms used. Table 9 shows the summary of *p*-value results for three performance measures for each dataset.

All datasets have the same decision to reject H_0 by looking at all *p*-values that were less than 0.05. Therefore, it can be concluded that There is at least one different algorithm based on the algorithms used for the results in terms of accuracy, ARI, and NMI, respectively. A post-hoc analysis is performed to determine which algorithm differs from the

Table 9
First-stage statistical test.

Dataset	*p*-value Accuracy	ARI	NMI	Decision
Iris	<0.001	<0.001	<0.001	Reject H_0
Flame	<0.001	<0.001	<0.001	Reject H_0
Jain	<0.001	<0.001	<0.001	Reject H_0
Wine	0.002	<0.001	<0.001	Reject H_0
Glass	<0.001	<0.001	<0.001	Reject H_0
Ecoli	<0.001	<0.001	<0.001	Reject H_0
Seed	<0.001	<0.001	<0.001	Reject H_0
Aggregation	<0.001	<0.001	<0.001	Reject H_0
Compound	<0.001	<0.001	<0.001	Reject H_0
Breast Tissue	<0.001	<0.001	<0.001	Reject H_0
Pima	0.004	<0.001	<0.001	Reject H_0
Room Occupancy	<0.001	<0.001	<0.001	Reject H_0
Rice	<0.001	<0.001	<0.001	Reject H_0
Wifi	<0.001	<0.001	<0.001	Reject H_0

R20 others. On each pair of groups, Dunn-Bonferroni post hoc tests were performed. Table 10 shows the results of the Dunn-Bonferroni test for each pair of groups for accuracy, ARI, and NMI. Significant differences between the two algorithms are represented at p-value $\leq$ 0.05 and marked "*." Conversely, the p-value $>$ 0.05, and the sign "-" indicate no significant difference between the two algorithms. Overall, the DP-PFCM algorithm has a significant difference from the other algorithms in terms of accuracy, ARI, and NMI for most of the benchmark datasets. The DP-PFCM algorithm is statistically superior to other algorithms since it has the lowest ranking out of all algorithms. This proves that using density peak clustering can solve the sensitivity of initial cluster centers, which is more effective than initializing random numbers.

4.3.2. *Second-stage experimental results*

DP-PFCM algorithm is the most effective algorithm in the first stage. Due to the difficulty in determining the parameters of the PFCM algorithm, metaheuristics are employed to determine the optimal combination of cluster centers and parameters that gives stable results. Table 11, Table 12, and Table 13 show the second-stage experimental results for the accuracy, ARI, and NMI, respectively. These results are the average values of 30 executions. The benchmark algorithms are denoted as follows: A1 represents DP-PFCM, A2 represents DP-GA-PFCM, A3 represents DP-PSO-PFCM, A4 represents DP-SCA-PFCM, and A5 represents DP-FBI-PFCM. Besides, this study also compares the result of the proposed method with two benchmark algorithms, i.e., genetic algorithm-based intuitionistic fuzzy weighted c-ordered-means algorithm (GA-IFWCOM) [14], and sine cosine algorithm-based fuzzy possibilistic *c*-ordered means algorithm (SCA-FPCOM) [28], which are denoted as A6 and A7, respectively in Tables 11-13. Moreover, the numbers in the parentheses () in these Tables show the corresponding rank of their result, as compared to seven algorithms. The same

R17 R23

Table 10
First-stage pairwise comparison.

Pair / Dataset		Accuracy K-means vs. DP-PFCM	FCM vs. DP-PFCM	PFCM vs. DP-PFCM	DP-FCM vs. DP-PFCM	AR K-means vs. DP-PFCM	FCM vs. DP-PFCM	PFCM vs. DP-PFCM	DP-FCM vs. DP-PFCM	NMI K-means vs. DP-PFCM	FCM vs. DP-PFCM	PFCM vs. DP-PFCM	DP-FCM vs. DP-PFCM
Iris		<0.001*	<0.001*	0.038*	0.002*	<0.001*	<0.001*	0.003*	0.002*	<0.001*	<0.001*	<0.001*	<0.001*
Flame		0.001*	0.033*	1.000	1.000	<0.001*	0.026*	1.000	1.000	<0.001*	0.028*	1.000	1.000
Jain		0.111	0.002*	0.956	<0.001*	0.399	0.002*	1.000	<0.001*	0.038*	0.008*	1.000	0.001*
Wine		0.002*	1.000	0.020*	0.072	0.001*	0.369	0.004*	0.011*	0.001*	0.250	0.003*	0.009*
Glass		<0.001*	<0.001*	1.000	0.027*	<0.001*	<0.001*	0.897	<0.001*	0.001*	<0.001*	1.000	<0.001*
Ecoli		1.000	0.745	0.337	<0.001*	0.011*	0.427	0.237	0.014*	0.827	1.000	0.038*	0.616
Seed		<0.001*	1.000	0.033*	1.000	<0.001*	0.080	0.007*	0.038*	<0.001*	1.000	0.006*	0.433
Aggregation		1.000	0.114	0.012*	1.000	<0.001*	0.014*	0.046*	<0.001*	1.000	0.004*	1.000	0.038*
Compound		0.006*	0.001*	0.005*	0.044*	0.002*	0.002*	0.043*	<0.001*	0.011*	0.004*	<0.001*	0.007*
Breast Tissue		<0.001*	0.038*	0.042*	<0.001*	<0.001*	<0.001*	<0.001*	0.002*	<0.001*	<0.001*	0.034*	<0.001*
Pima		0.010*	1.000	0.008*	0.458	<0.001*	1.000	1.000	1.000	<0.001*	1.000	1.000	0.919
Room Occupancy		<0.001*	<0.001*	<0.001*	<0.001*	<0.001*	<0.001*	<0.001*	<0.001*	<0.001*	<0.001*	<0.001*	<0.001*
Rice		<0.001*	<0.001*	<0.001*	<0.001*	<0.001*	<0.001*	<0.001*	<0.001*	<0.001*	<0.001*	<0.001*	<0.001*
Wifi		<0.001*	<0.001*	<0.001*	<0.001*	<0.001*	<0.001*	0.245	0.326	<0.001*	<0.001*	<0.001*	<0.001*
SUM	*	11	9	10	9	9	10	8	11	12	10	9	10
	–	3	5	4	5	2	4	6	3	2	4	5	4

Table 11
Second-stage accuracy experimental result.

Dataset	Average Accuracy (Rank) A1	A2	A3	A4	A5	A6	A7
Iris	0.908 (3)	0.914 (2)	0.899(6)	0.901 (5)	0.89 (7)	0.960 (1)	0.902 (4)
Flame	0.855 (2)	0.848 (4)	0.843 (6)	0.857 (1)	0.847 (5)	0.850 (3)	0.836 (7)
Jain	0.879 (5)	0.882 (1)	0.881 (2)	0.881 (2)	0.881 (2)	0.846 (7)	0.874 (6)
Wine	0.952 (5)	0.955 (4)	0.951 (6)	0.950 (7)	0.958 (3)	0.970 (1)	0.961 (2)
Glass	0.066 (7)	0.581 (2)	0.561 (4)	0.570 (3)	0.551 (5)	0.626 (1)	0.548 (6)
Ecoli	0.817 (4)	0.777 (6)	0.792 (5)	0.831 (2)	0.832 (1)	0.765 (7)	0.824 (3)
Seed	0.899 (3)	0.900 (2)	0.890 (6)	0.891 (4)	0.891 (4)	0.914 (1)	0.887 (7)
Aggregation	0.909 (2)	0.910 (1)	0.903 (4)	0.908 (3)	0.896 (6)	0.889 (7)	0.901 (5)
Compound	0.838 (5)	0.845 (3)	0.844 (4)	0.847 (2)	0.852 (1)	0.821 (7)	0.828 (6)
Breast Tissue	0.611 (2)	0.615 (1)	0.574 (5)	0.603 (3)	0.565 (7)	0.579 (4)	0.572 (6)
Pima	0.681 (2)	0.708 (1)	0.662 (6)	0.666 (5)	0.672 (4)	0.673 (3)	0.654 (7)
Room Occupancy	0.862 (6)	0.889 (1)	0.868 (4)	0.871 (3)	0.884 (2)	0.866 (5)	0.854 (7)
Rice	0.912 (6)	0.943 (2)	0.917 (5)	0.945 (1)	0.928 (3)	0.924 (4)	0.903 (7)
Wifi	0.945 (7)	0.962 (1)	0.952 (3)	0.952 (3)	0.947 (6)	0.960 (2)	0.951 (5)
Sum of Rank	**59**	**31**	**66**	**44**	**56**	**53**	**78**

R22 R21

Table 12
Second-stage ARI experimental result.

Dataset	Average ARI (Rank)						
	A1	A2	A3	A4	A5	A6	A7
Iris	0.759 (3)	0.774 (2)	0.740 (6)	0.744 (4)	0.723 (7)	0.89 (1)	0.742 (5)
Flame	0.501 (3)	0.482 (5)	0.469 (7)	0.507 (2)	0.481 (6)	0.68 (1)	0.487 (4)
Jain	0.569 (5)	0.575 (1)	0.574 (2)	0.572 (4)	0.573 (3)	0.543 (7)	0.566 (6)
Wine	0.858 (4)	0.865 (3)	0.857 (5)	0.852 (6)	0.877 (2)	0.852 (6)	0.889 (1)
Glass	0.194 (2)	0.183 (5)	0.172 (6)	0.190 (3)	0.161 (7)	0.532 (1)	0.184 (4)
Ecoli	0.419 (4)	0.352 (7)	0.359 (6)	0.435 (1)	0.430 (2)	0.367 (5)	0.428 (3)
Seed	0.729 (3)	0.730 (2)	0.704 (6)	0.707 (4)	0.706 (5)	0.826 (1)	0.692 (7)
Aggregation	0.683 (7)	0.695 (5)	0.701 (2)	0.700 (3)	0.723 (1)	0.698 (4)	0.694 (6)
Compound	0.547 (5)	0.555 (3)	0.554 (4)	0.557 (2)	0.562 (1)	0.532 (6)	0.500 (7)
Breast Tissue	0.393 (2)	0.370 (4)	0.383 (3)	0.320 (5)	0.292 (7)	0.482 (1)	0.308 (6)
Pima	0.115 (4)	0.166 (3)	0.010 (7)	0.102 (6)	0.106 (5)	0.355 (1)	0.233 (2)
Room Occupancy	0.739 (6)	0.816 (1)	0.793 (3)	0.79 (4)	0.802 (2)	0.764 (5)	0.721 (7)
Rice	0.853 (3)	0.856 (2)	0.832 (7)	0.857 (1)	0.844 (6)	0.848 (5)	0.850 (4)
Wifi	0.861 (7)	0.895 (1)	0.870 (5)	0.880 (2)	0.869 (6)	0.873 (4)	0.878 (3)
Sum of Rank	**58**	**44**	**69**	**47**	**60**	**48**	**65**

Table 13
Second-stage NMI experimental result.

Dataset	Average NMI (Rank)						
	A1	A2	A3	A4	A5	A6	A7
Iris	0.764 (2)	0.775 (1)	0.747 (5)	0.748 (4)	0.744 (7)	0.752 (3)	0.746 (6)
Flame	0.455 (2)	0.439 (3)	0.43 (5)	0.463 (1)	0.438 (4)	0.392 (7)	0.418 (6)
Jain	0.533 (3)	0.533 (3)	0.526 (6)	0.529 (5)	0.536 (1)	0.526 (6)	0.536 (1)
Wine	0.842 (5)	0.848 (4)	0.842 (5)	0.836 (7)	0.859 (3)	0.863 (2)	0.892 (1)
Glass	0.353 (4)	0.346 (5)	0.33 (7)	0.357 (3)	0.331 (6)	0.641 (1)	0.384 (2)
Ecoli	0.596 (3)	0.556 (5)	0.569 (4)	0.618 (2)	0.62 (1)	0.481 (7)	0.532 (6)
Seed	0.701 (2)	0.696 (3)	0.674 (7)	0.676 (5)	0.675 (6)	0.785 (1)	0.678 (4)
Aggregation	0.836 (3)	0.837 (1)	0.836 (3)	0.837 (1)	0.833 (6)	0.798 (7)	0.834 (5)
Compound	0.708 (6)	0.714 (2)	0.713 (4)	0.714 (2)	0.717 (1)	0.712 (5)	0.695 (7)
Breast Tissue	0.562 (2)	0.536 (5)	0.539 (3)	0.537 (4)	0.521 (6)	0.5982 (1)	0.508 (7)
Pima	0.07 (4)	0.105 (3)	0.057 (6)	0.055 (7)	0.058 (5)	0.246 (1)	0.182 (2)
Room Occupancy	0.843 (5)	0.875 (1)	0.853 (3)	0.847 (4)	0.839 (6)	0.864 (2)	0.833 (7)
Rice	0.876 (6)	0.903 (1)	0.886 (3)	0.897 (2)	0.882 (4)	0.852 (7)	0.881 (5)
Wifi	0.9 (4)	0.945 (1)	0.905 (3)	0.931 (2)	0.899 (5)	0.886 (7)	0.899 (5)
Sum of Rank	**51**	**38**	**64**	**49**	**61**	**57**	**64**

procedure is applied as in the first stage by calculating the sum of the rank values of each algorithm.

As can be seen from Table 11, Table 12, and Table 13, the sum of rank values for DP-GA-PFCM algorithm is the smallest one compared to other algorithms consistently in terms of accuracy, ARI, and NMI. Although for each dataset the first ranking algorithm differs on each performance measure, DP-GA-PFCM algorithm consistently performs better than other algorithms based on accuracy, ARI, and NMI. It can be

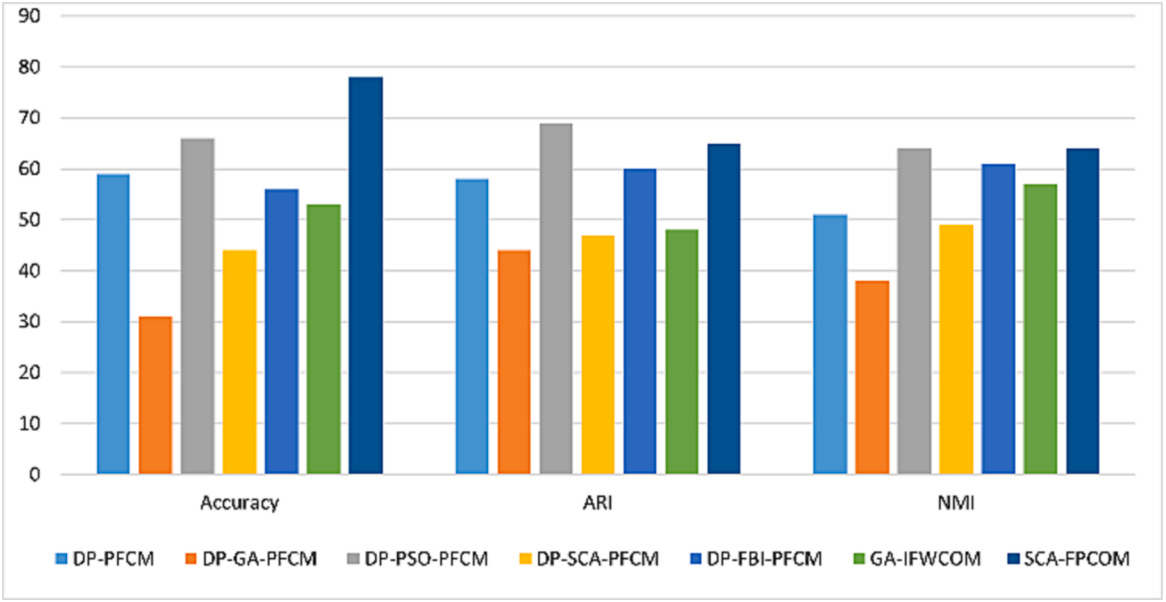


Fig. 4. The sum of ranks in the second stage of the experiment.

concluded that DP-GA-PFCM can perform better results. Similarly in the first stage of the experiment, the sum of ranks of all the tested algorithms is also presented in Fig. 4 for better visualizatio. It can be seen that the proposed DP-GA-PFCM rank #1 for the three validation indices, i.e., accuracy, ARI, and NMI in terms of sum of ranks for all tested datasets. DP-SCA-PFCM ranks the 2nd position while the 3rd one is GA-IFWCOM algorithms.

The second-stage statistical test was performed to determine whether there is a difference based on the algorithms used which are DP-PFCM algorithm, DP-GA-PFCM algorithm, DP-PSO-PFCM algorithm, DP-SCA-PFCM algorithm, and DP-FBI-PFCM algorithm. Similar to the first-stage result, there is at least one different algorithm based on the algorithms used. A post-hoc test is conducted to identify which algorithms have differences. Table 14 shows the results of the Dunn-Bonferroni test for each pair of groups in terms of accuracy, ARI, and NMI. The result shows that the DP-GA-PFCM is significantly better than the other algorithms.

4.4. Analysis and discussion

Eight of the fourteen datasets contain outliers, which include Flame, Wine, Glass, Ecoli, Compound, Seed, Breast Tissue, and Pima. As shown in Table 6, Table 7, and Table 8, the PFCM algorithm can overcome the presence of outliers by optimizing the accuracy, ARI, and NMI performance of PFCM and Dp-PFCM algorithms on the Iris, Flame, Wine, Glass, Seed, Compound, Breast Tissue, and Pima datasets. Furthermore, the sum of rank values indicates the use of the PFCM algorithm has top-performing accuracy, ARI, and NMI. Although the Iris dataset contains no outliers, the density peak algorithm leads to better results. Additionally, Jain and Aggregation datasets do not contain any outliers, so PFCM is not able to yield better results. Even though Ecoli has outliers, its performance does not seem to be satisfactory due to three classes with less than or equal to five members. This condition makes it difficult for both the density peak and the PFCM algorithms to give better results.

Overall, the DP-GA-PFCM algorithm has the most significant differences from other algorithms based on accuracy, ARI, and NMI. Since DP-GA-PFCM algorithm has the lowest ranking among other algorithms, as a result, DP-GA-PFCM algorithm is superior to other algorithms. By using the genetic algorithm, the DP-GA PFCM algorithm can be optimized for parameters and cluster centers, providing better results. It can be seen that GA, PSO, SCA, and FBI algorithms are essentially population-based metaheuristics. Although those metaheuristics can handle both discrete and continuous problems, GA is more effective for handling discrete problems, while PSO, SCA, and FBI algorithms are better at dealing with continuous problems [22–24]. From the overall results, GA-based gives better results than other metaheuristics. For instance, GA-based DP-PFCM is better as compared with PSO-based DP-PFCM, SCA-based DP-PFCM, FBI-based DP-PFCM, or GA-IFWCOM is also better than SCA-FPCOM. Here, genetic operations play an important role in good exploration and exploitation in the search space. Furthermore, PSO, SCA, and FBI algorithms tend to be difficult to find optimal solutions because the MinMax Scaler that changes cluster centers in the range 0 to 1 makes the individual update method on these algorithms less effective because the updated value cannot be less than zero or>1. Since this study deals with continuous problems, the PSO, SCA, and FBI algorithms are more advantageous, but genetic operations are more effective and efficient to provide optimal results. The result is proven through the statistical test in Table 14 where DP-GA-PFCM perform significantly better in most tested datasets. Moreover, Table 15 shows the standard deviation of each algorithm after 30 runs in terms of accuracy, ARI, and NMI. The result proves that the proposed DP-GA-PFCM yields more reliable and robust results with the lowest standard deviation.

Besides, the computational time, which is the time required to execute the algorithm used, is also analyzed. Table 16 shows the result of the computational time (in seconds) of each algorithm on each dataset.

Table 14
Second-stage pairwise comparison.

Pair	Accuracy							ARI							NMI						
	A2 vs. A1	A2 vs. A3	A2 vs. A4	A2 vs. A5	A2 vs. A6	A2 vs. A7		A2 vs. A1	A2 vs. A3	A2 vs. A4	A2 vs. A5	A2 vs. A6	A2 vs. A7		A2 vs. A1	A2 vs. A3	A2 vs. A4	A2 vs. A5	A2 vs. A6	A2 vs. A7	
Dataset																					
Iris	0.07	0.00*	0.00*	0.00*	0.00*	0.00*		0.8	0.00*	0.00*	0.00*	0.00*	0.00*		0.26	0.00*	0.00*	0.00*	0.00*	0.00*	
Flame	0.04*	0.03*	0.00*	1.00	0.00*	0.00*		0.04*	0.03*	0.00*	1.000	0.00*	0.00*		0.03*	0.03*	0.00*	1.00	0.00*	0.00*	
Jain	0.01*	1.00	1.00	1.00	0.00*	0.00*		0.01*	1.00	1.00	1.00	0.00*	0.00*		1.00	0.01*	0.56	1.00	0.00*	0.00*	
Wine	0.69	0.21	0.02*	0.16	0.00*	0.00*		1.00	1.00	0.01*	0.02*	0.14	0.00*		1.00	1.00	0.01*	0.01*	0.00*	0.00*	
Glass	0.00*	0.00*	0.02*	0.00*	0.00*	0.00*		0.04*	0.17	1.00	0.00*	0.00*	0.00*		1.00	0.01*	1.00	0.02*	0.00*	0.00*	
Ecoli	0.00*	0.72	0.00*	0.00*	0.00*	0.00*		0.00*	1.00	0.00*	0.00*	0.00*	0.00*		0.00*	0.85	0.00*	0.00*	0.00*	0.00*	
Seed	1.00	0.00*	0.00*	0.00*	0.00*	0.00*		1.00	0.00*	0.00*	0.00*	0.00*	0.00*		1.00	0.00*	0.00*	0.00*	0.00*	0.01*	
Aggregation	1.00	0.03*	1.00	0.00*	0.00*	0.00*		0.01*	1.00	1.00	0.00*	0.48	0.00*		1.00	1.00	1.00	0.00*	0.00*	0.00*	
Compound	0.03*	1.00	1.00	0.18	0.00*	0.00*		0.01*	1.00	1.00	0.47	0.00*	0.00*		0.04*	1.00	1.00	1.00	0.00*	0.00*	
Breast Tissue	1.00	0.00*	0.01*	0.00*	0.00*	0.00*		0.00*	0.516	0.01*	0.00*	0.00*	0.00*		0.00*	1.00	1.00	0.01*	0.00*	0.00*	
Pima	0.00*	0.00*	0.00*	0.00*	0.00*	0.00*		0.00*	0.00*	0.00*	0.00*	0.00*	0.00*		0.00*	0.00*	0.00*	0.00*	0.00*	0.00*	
Room Occupancy	0.00*	0.00*	0.00*	0.00*	0.00*	0.00*		0.00*	0.00*	0.00*	0.00*	0.00*	0.00*		0.00*	0.00*	0.00*	0.00*	0.02*	0.00*	
Rice	0.00*	0.00*	0.00*	0.00*	0.00*	0.00*		0.32	0.00*	0.13	0.00*	0.00*	0.00*		0.00*	0.00*	0.25	0.00*	0.00*	0.00*	
Wifi	0.00*	0.00*	0.00*	0.00*	0.16	0.00*		0.00*	0.00*	0.00*	0.00*	0.00*	0.00*		0.00*	0.00*	0.00*	0.00*	0.00*	0.00*	
SUM	* 9	10	11	10	13	14		10	7	9	11	12	14		8	9	8	11	14	14	
	– 5	4	3	4	1	0		4	7	5	3	2	0		6	5	6	3	0	0	

R19 **Table 15**
Standard deviation of each algorithm.

Algorithm	Accuracy	ARI	NMI
K-Means	0.029	0.021	0.133
FCM	0.025	0.018	0.102
PFCM	0.020	0.016	0.081
DP-FCM	0.016	0.014	0.062
DP-PFCM	0.016	0.017	0.073
DP-GA-PFCM	0.012	0.010	0.059
DP-PSO-PFCM	0.016	0.014	0.087
DP-SCA-PFCM	0.021	0.018	0.075
GA-IFWCOM	0.028	0.032	0.068
SCA-FPCOM	0.015	0.026	0.064

Based on the result, adding the DPC method will take a slightly longer computational time than conventional methods such as *K*-means algorithm, FCM algorithm, and PFCM algorithm. However, the result in the first stage of the experiment proved that the DP-PFCM is significantly better than other traditional methods without adding *meta*-heuristics. Furthermore, combining a metaheuristic with clustering will take a longer time than other traditional methods because the metaheuristic performs iteratively to find optimal solutions for PFCM algorithm. The computational time of DP-GA-PFCM is also better than those of GA-IFWCOM and SCA-FPCOM. In addition, the FBI algorithm is the algorithm with the longest computational time due to the complexity of the procedure for updating individuals in its population. Although the time spent between GA, PSO, and SCA is not much different, GA can provide better results, as shown in the second stage of the experiment. As a result, GA is more effective than PSO, SCA, and FBI algorithms.

5. Case study

R25 In this case study, a customer segmentation dataset from Mendeley Data was implemented. This retail business dataset consists of 1288 records of customers with five variables including the customer ID, last purchase date, recency, frequency, and monetary. For cluster analysis, the last three variables, recency, frequency, and monetary (RFM) have an important role in customer segmentation. RFM analysis is one way to identify customer behavior. The recency refers to the time since the customer's last transaction. The smaller the recency, the more the customer has recently made a transaction. Customer frequency refers to the number of transactions a customer makes in a given period. The greater the frequency, the greater the number of transactions made by

R24 **Table 16**
Computational time (second).

	K-means	FCM	PFCM	DP-FCM	DP-PFCM	DP-GA-PFCM	DP-PSO-PFCM	DP-SCA-PFCM	DP-FBI-PFCM	GA-IFWCOM	SCA-FPCOM
Iris	0.02	1.64	0.55	2.13	2.02	48.4	51.4	48.6	198.5	221.4	178.2
Flame	0.01	1.58	0.69	2.45	3.07	77.0	65.4	62.1	234.1	266.8	243.1
Jain	0.01	2.41	1.14	4.29	6.08	117.9	107.1	99.2	391.2	357.3	386.5
Wine	0.01	2.49	0.86	3.04	2.71	76.5	83.3	73.5	369.2	482.5	563.7
Glass	0.04	7.05	1.75	8.48	5.56	157.2	169.1	148.1	962.3	1823.4	1783.4
Ecoli	0.05	10.5	2.93	14.5	12.0	251.7	277.4	238.2	1052.8	2290.1	3201.6
Seed	0.02	2.47	0.87	3.28	3.17	78.1	81.7	73.4	249.2	362.3	328.4
Aggregation	0.04	23.7	6.40	30.6	47.9	577.3	672.3	558.8	2220.6	2531.6	2302.9
Compound	0.03	9.99	2.49	11.5	12.9	215.5	187.9	214.2	1281.6	986.7	862.2
Breast Tissue	0.04	2.64	0.80	3.42	2.01	72.4	77.5	63.6	266.7	899.3	783.1
Pima	0.23	7.13	2.99	15.5	18.5	256.7	260.1	231.7	951.8	1024.9	1102.3
Room Occupancy	3.24	18.64	18.22	30.2	33.57	451.2	464.1	398.0	1104.2	1124.2	938.6
Rice	1.48	14.79	12.05	16.41	20.14	326.3	347.5	302.6	780.5	823.5	643.5
Wifi	1.12	12.90	9.87	13.69	14.08	298.6	320.9	285.7	737.1	798.3	624.9

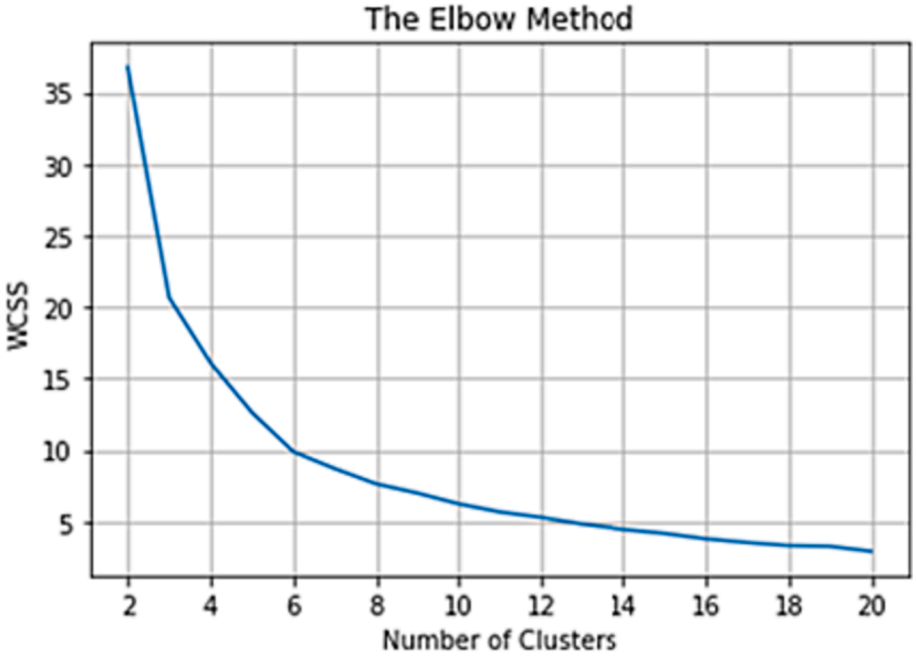


Fig. 5. The elbow method output.

Table 17
RFM categorization.

Variables	Small	Medium	High
Recency	< 251	251–320	> 320
Frequency	< 2	2–15	> 15
Monetary	< 16	16–45	> 45

customers. A customer's monetary amount is the amount of money they issue over a specified period. The bigger the monetary, the more money the customer will spend. RFM analysis has been widely used in applications such as retail to determine marketing strategies and has proven to be a powerful technique to rank customers based on prior transaction history.

5.1. Clustering result

R25 Since this study proposes an algorithm to handle noise and outlier data, the data preprocess is to eliminate data containing missing values and remove negative values. Consequently, the total of data used is 1281 records. The variables used for cluster analysis are recency, frequency, and monetary. Moreover, because the units of variables are different, the Min-Max Scaler is used to transform the values of each variable in the R26 range of 0 to 1. The proposed algorithm required two input parameters, the number of clusters and the maximum number of iterations. The number of clusters is determined by the Elbow method shown in Fig. 5. From the graph, the number of clusters used is 6 clusters. Also, the maximum number of iterations used is 100.

5.2. Analysis and discussion

Market segmentation in the industry is commonly discussed in many studies using various approaches, including clustering. Rahim et al. [29] explained that by using machine learning, high accuracy was obtained according to customer membership. The RFM model is applied to determine customer behavior in the retail industry. However, in the tourism industry, machine learning can also classify customers based on reviews for decision-making related to green policies and environmental-friendly practices.

The computational time required in the case study dataset using the DP-GA-PFCM algorithm is 1017.32 s. Since GA is superior to the other metaheuristics, it can be said that the way GA updates its individual by crossover and mutation is more effective and efficient for obtaining a combination of parameter and cluster centers compared to PSO by using velocity, SCA with the sine and cosine equations, and FBI algorithm with the "investigation" and "pursuit" to get the targeted individual. The transformation within the GA calculation can anticipate getting caught in local optima and makes a difference to extend its search scope [30]. To ease the identification of customer categories for each cluster, the variables used are divided into three categories: small, medium, and high as shown in Table 17.

The graph of each customer member in each cluster is shown in Fig. 6. There are six clusters with different colors and characteristics. The detailed characteristics of each cluster are illustrated in Table 18. In the retail business, good customers are indicated by small recency, high frequency, and high monetary to get more profit. Based on the result, the most potential customers are represented by Cluster 1 and Cluster 5. They have small recency, and medium frequency, and differ only in the amount of money spent. It means recently they made transactions with

Table 18 R26
Characteristics of each cluster.

Cluster	Recency	Frequency	Monetary	Members
Cluster 1	Small	Medium	High	196
Cluster 2	High	Medium	Medium	341
Cluster 3	Medium	Medium	Small	194
Cluster 4	Medium	Medium	Medium	108
Cluster 5	Small	Medium	Medium	334
Cluster 6	Medium	Medium	Medium	108

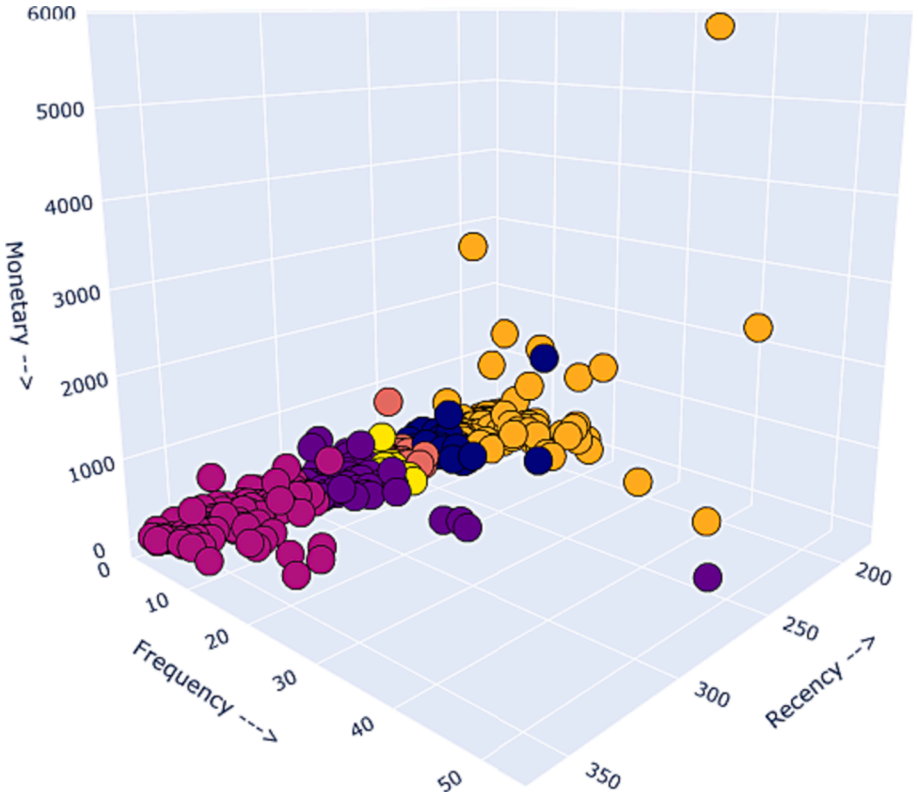


Fig. 6. Clustering result.

several transactions between 2 and 15 times. Cluster 1 can be said to be the best customer because the expenses are greater than Cluster 5. These clusters need to be appreciated by giving rewards to them for being the customers who have contributed the most to the company. Cluster 2 is an occasional customer which is indicated by the high recency and medium frequency. Accordingly, they only occasionally come to buy some stuff. Thus, the company can provide special offers such as discounts to increase their visits. Saving customers are represented in cluster 3. They only spent a small amount of money which means only buying things that are not too expensive such as daily or monthly needs. Offering a membership and product recommendation can lead to upselling the product. Cluster 4 and Cluster 6 are identical. They have medium recency, medium frequency, and medium monetary. It can be said that they are "need attention" customers. They are in an intermediate condition, the right strategy can be to loyal customers. Otherwise, the bad strategy will lead to a loss of customers. Even though they are the same, after being seen in more detail, Cluster 4 has a customer segment that will be more inclined to be loyal customers. Some of them have lower recency than cluster 6. So, companies need to provide strategies such as promotions to retain customers in Cluster 4 and attract customers in Cluster 6. Cluster 5 is a stable customer or can be raised to potential loyalists, they are recent customers with average frequency and spent a good amount of money. With a willingness to pay, consider premium offers and offer luxury products.

In summary, the results indicate that each segment should be implemented a different marketing strategy. The right and specific strategy can influence the customer behavior of each cluster. In the end, the company will get the maximum profit.

6. Conclusion

This study proposed an algorithm that combines density peak clustering with possibilistic fuzzy c-means algorithm and GA. The proposed DP-GA-PFCM algorithm can overcome (1) the presence of noise and outlier data and (2) the sensitivity of initial cluster centers. These computational results have proved that the density peak can produce initial cluster centers more efficiently instead of using random numbers. In addition, using a possibilistic concept can handle the influence of noise and outlier data. The DP-PFCM algorithm which integrates with metaheuristics, such as GA, PSO, SCA, and FBI algorithms, to find the optimal combination of parameters and cluster centers can yield better performance. In addition, the result shows that GA can enhance the clustering performance more than the others. According to the result of the DP-GA-PFCM algorithm for the case study, there are six customer clusters with their respective characteristics. Also, this study provides some strategies to maintain and attract customers.

Though the experimental results of the proposed algorithm have shown that it gives better performance, it still can be improved in some ways. The DP-GA-PFCM algorithm requires the number of clusters as an input, it is better if the algorithm can automatically determine the number of clusters itself. Although GA can provide better results, future research could implement other metaheuristics or modify the existing metaheuristics to reduce the computational time and increase the performance. Besides, the study can be extended to validate the results of high-dimensional datasets. Moreover, the actual loss with statistical significance can be analyzed to evaluate the clustering performance in future research.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research is funded by Funds for Science and Technology Development of the University of Danang under project number B2020-DN02-83. This support is much appreciated.

References

[1] A. Ahani, M. Nilashi, O. Ibrahim, L. Sanzogni, S. Weaven, Market segmentation and travel choice prediction in Spa hotels through TripAdvisor's online reviews, Int. J. Hosp. Manag. 80 (2019) 52–77.
[2] J. Zhou, L. Zhai, A. Pantelous, Market segmentation using high-dimensional sparse consumers data, Expert Syst. Appl. 145 (2020) 1–17.
[3] H. Zhou, R. Norman, K. Kelobonye, J. Xia, B. Hughes, G. Nikolova, T. Falkmer, Market segmentation approach to investigate existing and potential aviation markets, Transp. Policy 99 (2020) 120–135.
[4] D. Moussa, N. Eissa, H. Abounaser, A. Badr, Design of novel metaheuristic techniques for clustering, IEEE Access 6 (2018) 77350–77358.
[5] M. Rezaei, Improving a centroid-based clustering by using suitable centroids from another clustering, J. Classif. 37 (2) (2020) 352–365.
[6] Y. Wang, D. Wang, Y. Zhou, X. Zhang, C. Quek, VDPC: Variational density peak clustering algorithm, Inf. Sci. 621 (2023) 627–651.
[7] Y. Li, L. Sun, Y. Tang, DPC-FSC: An approach of fuzzy semantic cells to density peaks clustering, Inf. Sci. 616 (2022) 88–107.
[8] X.-y. Liu, J.-C. Fan, Z.-W. Chen, Improved fuzzy c-means algorithm based on density peak, Int. J. Mach. Learn. Cybern. 11 (3) (2020) 545–552.
[9] W. Tong, S. Liu, X.Z. Gao, A density-peak-based clustering algorithm of automatically determining the number of clusters, Neurocomputing (2020) 1–12.
[10] N. Pal, K. Pal, J. Keller, J. Bezdek, A possibilistic fuzzy c-means clustering algorithm, IEEE Trans. Fuzzy Syst. 13 (4) (2005) 517–530.
[11] R. Krishnapuram, J.M. Keller, A possibilistic approach to clustering, IEEE Trans. Fuzzy Syst. 1 (2) (1993) 98–110.
[12] D. Zeebaree, H. Haron, A. Abdulazeez, S. Zeebaree, Combination of K-means clustering with genetic algorithm: A review, Int. J. Appl. Eng. Res. 12 (24) (2017) 14238–14245.
[13] L. Zhu, Safety detection algorithm in sensor network based on ant colony optimization with improved multiple clustering algorithms, Saf. Sci. 118 (2019) 96–102.
[14] R.J. Kuo, J.-Y. Lin, T.P.Q. Nguyen, An application of sine cosine algorithm-based fuzzy possibilistic c-ordered means algorithm to cluster analysis, Soft Comput. 25 (5) (2021) 3469–3484.
[15] A. Chehouri, R. Younes, J. Khoder, J. Perron, A. Ilinca, A selection process for genetic algorithm using clustering analysis, Algorithms 10 (123) (2017) 1–15.
[16] A. Golmohammadi, H. Bani-Asadi, H. Zanjani, H. Tikani, A genetic algorithm for preemptive scheduling of a single machine, Int. J. Ind. Eng. Comput. 7 (2016) 607–614.
[17] M. Du, S. Ding, X. Xu, Y.u. Xue, Density peaks clustering using geodesic distances, Int. J. Mach. Learn. Cybern. 9 (8) (2018) 1335–1349.
[18] X. Chen, D. Li, H. Li, An improved type-2 possibilistic fuzzy c-means clustering algorithm with application for MR image segmentation, International Journal of Signal Processing, Imgae Processing and Pattern Recognition 9 (11) (2016) 363–370.
[19] A. Hassanat, K. Almohammadi, E. Alkafaween, E. Abunawas, A. Hammouri, V.B. S. Prasath, Choosing Mutation and Crossover Ratios for Genetic Algorithms—A Review with a New Dynamic Approach, Information. 10 (12) (2019) 390.
[20] L. Haldurai, T. Madhubala, R. Rajalakshmi, A study on genetic algorithm and its applications, Int. J. Comput. Sci. Eng. 4 (10) (2016) 139–143.
[21] E. Michielssen, S. Ranjithan, R. Mittra, Optimal multilayer filter design using real coded genetic algorithms, IEE Proceedings J-Optoelectronics 139 (6) (1992) 413.
[22] Y. Xue, B. Xue, M. Zhang, Self-adaptive particle swarm optimization for large-scale feature selection in classification, ACM Trans. Knowl. Discov. Data (TKDD) 13 (5) (2019) 1–27.
[23] M.A. Tawhid, P. Savsani, Discrete sine cosine algorithm (DSCA) with local search for solving traveling salesman problem, Arab. J. Sci. Eng. 44 (4) (2019) 3669–3679.
[24] J. Chou, N. Nguyen, FBI inspired meta-optimization, Appl. Soft Comput. 93 (2020) 1–28.
[25] R.J. Kuo, P. Amornnikun, T.P.Q. Nguyen, Metaheuristic-based possibilistic multivariate fuzzy weighted c-means algorithms for market segmentation, Appl. Soft Comput. 96 (2020) 1–14.
[26] R.J. Kuo, Y.R. Zheng, T.P.Q. Nguyen, Metaheuristic-based possibilistic fuzzy k-modes algorithms for categorical data clustering, Inf. Sci 557 (2021) 1–15.
[27] X.F. Song, Y. Zhang, Y.N. Guo, X.Y. Sun, Y.L. Wang, Variable-size cooperative coevolutionary particle swarm optimization for feature selection on high-dimensional data, IEEE Trans. Evol. Comput. 24 (5) (2020) 882–895.
[28] R.J. Kuo, C.K. Chang, T.P.Q. Nguyen, T.W. Liao, Application of genetic algorithm-based intuitionistic fuzzy weighted c-ordered-means algorithm to cluster analysis, Knowl. Inf. Syst 63 (7) (2021) 1935–1959.
[29] M.A. Rahim, M. Mushafiq, S. Khan, Z.A. Arain, RFM-based repurchase behavior for customer segmentation, J. Retail. Consum. Serv. 61 (2021) 1–9.
[30] R.J. Kuo, T.C. Lin, F.E. Zulvia, C.Y. Tsai, A hybrid metaheuristic and kernel intuitionistic fuzzy c-means algorithm for cluster analysis, Appl. Soft Comput. 67 (2018) 299–308.

Đánh giá khả năng tái tạo DP-GA-PFCM

Phụ lục do Codex biên soạn - 04/10/2026

Đánh giá tổng hợp: 65%. Điểm đánh giá chuyên gia về mức độ đầy đủ của thông tin để viết lại mô hình và tổ chức so sánh. Không phải xác suất thống kê, mức khớp kết quả đã đo hoặc bảo đảm tái hiện số liệu đến 3 chữ số thập phân.

Có thể viết bản triển khai chạy được, bám cấu trúc DPC → GA → PFCM. Các lựa chọn chưa công bố phải ghi là giả định. Chưa đủ cơ sở gọi bản triển khai bất kỳ là bản tương đương chính xác với code tác giả.

Cách đọc bản PDF này

13 trang đầu giữ nguyên nội dung bài báo gốc. Các vùng tô màu và nhãn R01-R27 là bổ sung của người đánh giá. Trang 14 trở đi là phân tích tiếng Việt. Bấm highlight trong trình đọc hỗ trợ annotations để xem ghi chú; nếu không mở popup được, tra mã Rxx ở phụ lục.

- ĐỎ - mâu thuẫn / lỗi ký hiệu cần xác nhận
- CAM - thiếu chi tiết triển khai
- VÀNG - thiếu thông tin đối chiếu thí nghiệm

Với thông tin hoàn toàn vắng mặt, highlight nằm ở đoạn gần nhất đang mô tả thành phần liên quan. Nó không có nghĩa thông tin thiếu đã xuất hiện trong đoạn đó. Lỗi ký hiệu/bảng chỉ phản ánh tài liệu in; không suy ra code tác giả cũng sai.

Phạm vi kiểm tra

Đọc toàn bộ 13 trang PDF gốc, kiểm tra trực quan công thức, Figure 1-2, Tables 1-16 và case study. Đối chiếu thêm UCI Glass và tài liệu NMI chính thức. Chưa chạy mô hình, chưa có code tác giả/raw results/manifest dữ liệu. Điểm số dựa trên tài liệu, không dựa trên benchmark mới.

Cách tính điểm 65/100

Hạng mục	Trọng số	Điểm	Cơ sở đánh giá
Luồng thuật toán và DPC	20	19	Có 3 giai đoạn, rho/delta, top-k, MinMax; dc và tie còn thiếu.
Công thức/cấu hình PFCM	20	12	Có công thức; thiếu K/gamma, sửa ký hiệu, tol và ổn định số.
Tối ưu bằng GA	20	10	Có chromosome/operator; fitness, bounds, init, mutation chưa đủ.
Dữ liệu và tiền xử lý	15	9	Có tên,N,F,K; thiếu file/version/schema/cleaning log.
Đánh giá và so sánh	15	12	Có metric/30 runs/kiểm định; thiếu raw, rule nhãn/SD/rank/baseline.
Code/môi trường/truy vết	10	3	Có Python/OS/CPU/RAM tổng quát; thiếu repo/seed/versions/log.
Tổng	100	65	Mức đủ thông tin ước lượng

Quy ước chấm điểm

Cho điểm phần thông tin đã có, trừ điểm khi thiếu lựa chọn có thể đổi kết quả. Trọng số phản ánh ảnh hưởng đến mục tiêu tái tạo và so sánh. Đây không phải rubric chuẩn hóa của một tổ chức; không diễn giải 65% với độ chính xác thống kê.

Ba mức tái tạo cần phân biệt

Viết lại ý tưởng và chạy được: khả thi với hồ sơ giả định rõ ràng và xử lý số học đầy đủ.

Tạo bảng so sánh cùng phương pháp: khả thi có điều kiện sau khi xác định dữ liệu/baseline/metric; số liệu có thể khác bài.

Khớp số liệu, thứ hạng, p-value và thời gian gốc: chưa đủ thông tin để cam kết hoặc gán một xác suất đáng tin cậy. Cần code/config/seed/raw output của tác giả.

Ưu tiên trước khi viết code

R01 fitness; R02 miền tham số; R03 K/gamma; R04-R07 trạng thái/operator GA; R08 mutation rate; R14-R16 dữ liệu/baseline; R18 metric. R21 là ô nghi lỗi in, cần cờ không dùng làm đích khớp tuyệt đối khi chưa xác nhận.

Các đoạn thiếu hoặc chưa rõ

Vị trí bên dưới là số trang bài gốc (1-13). Các cách xử lý là đề xuất cho bản tái tạo, không phải mô tả mặc định về code tác giả.

R01 - Hàm fitness của GA chưa xác định nhất quán

Vị trí: Trang 4, Step 6; đối chiếu Eq. (6),(11)

Thông tin trong bài: Step 6 gọi PFCM nhưng dẫn Eq. (6).

Thiếu / chưa rõ: Eq. (6) là mục tiêu FCM; mục tiêu PFCM nằm ở Eq. (11). Within-cluster sum of squared errors còn có thể được hiểu là SSE của nhãn cứng. Chưa rõ fitness lấy trước hay sau một bước PFCM.

Ảnh hưởng: Các cách tính có thể chọn tâm và tham số rất khác nhau.

Cách xử lý: Xin code/công thức fitness. Nếu chưa có, chạy riêng J_FCM, J_PFCM và SSE; không coi một cách hiểu là bản gốc.

R02 - Miền tìm kiếm a, b, w và eta bị thiếu

Vị trí: Trang 4, mô tả nhiễm sắc thể

Thông tin trong bài: Bài chỉ yêu cầu các giá trị lớn hơn 0.

Thiếu / chưa rõ: Không có bounds hoặc phân phối khởi tạo. Công thức chứa 1/(w-1), 1/(eta-1), nên >0 chưa đủ cho cập nhật PFCM thông thường; w=1 hoặc eta=1 không xác định.

Ảnh hưởng: Không tái hiện được search space. Nếu dùng J_PFCM mà không kiểm soát thang a,b, GA có thể ưu tiên giảm thang trọng số thay vì cải thiện cụm.

Cách xử lý: Xin bounds và repair. Ràng buộc w,eta>1 là giả định cần công khai.

R03 - K và lịch cập nhật gamma chưa được quy định

Vị trí: Trang 3, Eq. (9); trang 4, đầu cột trái

Thông tin trong bài: Eq. (9) dùng gamma_k = K nhân một tỉ số.

Thiếu / chưa rõ: K không được giải thích hoặc gán giá trị tại đây; không tự đồng nhất với số cụm K ở Table 1. Không rõ gamma tính một lần, mỗi cá thể GA hay mỗi vòng PFCM.

Ảnh hưởng: Gamma quyết định typicality và ảnh hưởng của điểm ngoại lai.

Cách xử lý: Xin K, lịch cập nhật gamma và xử lý gamma=0; kiểm tra độ nhạy nếu đặt giả định.

R04 - Chưa rõ cách tạo 80 cá thể từ tâm DPC

Vị trí: Trang 4, đoạn khởi tạo và Step 5

Thông tin trong bài: Tâm quần thể ban đầu được lấy từ DPC.

Thiếu / chưa rõ: Không nói sao chép tâm, thêm nhiễu hay trộn tâm ngẫu nhiên. Thiếu biên độ nhiễu, cách sinh tham số và căn chỉnh thứ tự tâm giữa cá thể.

Ảnh hưởng: Quần thể ban đầu quyết định tính đa dạng; hoán vị tâm có thể làm phép lai tạo tâm trùng.

Cách xử lý: Công bố công thức sinh cá thể và thứ tự gene; lưu quần thể ban đầu theo seed.

R05 - Một bước PFCM bên trong GA chưa có trình tự chi tiết

Vị trí: Trang 3, cuối mục 3.2

Thông tin trong bài: PFCM chạy một iteration cho mỗi GA generation.

Thiếu / chưa rõ: Chưa rõ một bước cho từng chromosome hay cho generation; thứ tự mu,gamma,t,V chưa ghi. Tâm sau PFCM có ghi ngược vào chromosome hay chỉ dùng đánh giá?

Ảnh hưởng: Có/không ghi ngược tâm tạo hai thuật toán lai khác nhau.

Cách xử lý: Cần pseudocode cấp cá thể, trạng thái đầu vào/đầu ra và tính xác định khi đánh giá lại cùng cá thể.

R06 - Roulette wheel thiếu phép chuyển loss thành xác suất

Vị trí: Trang 4, Step 7

Thông tin trong bài: Selection dùng roulette wheel.

Thiếu / chưa rõ: Không có công thức xác suất cho bài toán tối thiểu hóa, epsilon khi loss=0, elitism, replacement hoặc best-so-far.

Ảnh hưởng: Dùng trực tiếp loss làm xác suất sẽ ưu tiên nghiệm kém; replacement khác nhau đổi hội tụ.

Cách xử lý: Xin công thức selection và elitism. Nếu dùng 1/(loss+epsilon), ghi là quyết định của bản tái tạo.

R07 - Chi tiết operator GA và xử lý biên chưa đủ

Vị trí: Trang 4, Eq. (14)-(16)

Thông tin trong bài: Mutation range 10%; precision k thuộc {4,...,20}.

Thiếu / chưa rõ: Không chọn precision k cụ thể; range 10% theo bounds hay bước tuyệt đối? Thiếu xác suất theo gene/cá thể, r dùng chung/từng gene, và repair tâm vượt [0,1].

Ảnh hưởng: Tham số và tọa độ tâm có thang khác nhau; phép đột biến cho hiệu ứng khác nhau.

Cách xử lý: Xin operator config và repair. Phân biệt mutation range 10% với mutation probability 0.1.

R08 - Mutation rate không khớp phần tuning

Vị trí: Trang 5, Table 2; trang 6, Tables 3,5

Thông tin trong bài: Tuning thử 0.01 và 0.03; Table 5 ghi 0.1.

Thiếu / chưa rõ: 0.1 nằm ngoài hai mức tuning. Từ số in ở Table 3, cột crossover=0.8, mutation=0.01 có trung bình cao nhất.

Ảnh hưởng: Không biết bảng cuối dùng 0.01,0.03 hay 0.1; 1% và 10% khác đáng kể.

Cách xử lý: Xin đính chính; chạy 0.01 và 0.1 thành hai config riêng, không tự sửa bài.

R09 - Ngưỡng dừng và ngân sách PFCM chưa rõ

Vị trí: Trang 4, Step 13 và Algorithm 1; Table 5

Thông tin trong bài: Dừng theo số vòng hoặc accepted threshold.

Thiếu / chưa rõ: Thiếu epsilon, tuyệt đối/tương đối và mục tiêu để xét hội tụ. Pseudocode chỉ có MAX_ITER. 100 vòng ở Table 5 dành cho metaheuristic, chưa xác nhận riêng PFCM cuối.

Ảnh hưởng: Ảnh hưởng kết quả và thời gian tính toán.

Cách xử lý: Tách ga_generations, pfcm_max_iter, tol và tiêu chí hội tụ; xin config gốc.

R10 - Chưa có quy tắc lấy nhãn cứng

Vị trí: Trang 4, cuối Algorithm 1

Thông tin trong bài: Report L as a set of cluster labels.

Thiếu / chưa rõ: Không nói argmax(mu), argmax(t), trọng số kết hợp hay tâm gần nhất; thiếu tie rule, cụm rỗng và xử lý điểm có typicality thấp ở mọi cụm.

Ảnh hưởng: Metric và số khách mỗi cụm đều phụ thuộc rule nhãn.

Cách xử lý: Xin rule; argmax(mu) chỉ là giả định. Lưu mu,t để kiểm tra các lựa chọn.

R11 - Cutoff distance 2% chưa đủ xác định dc

Vị trí: Trang 5, mục 4.2; trang 2, Eq. (2)-(3)

Thông tin trong bài: A cutoff distance of 2% is set for DPC.

Thiếu / chưa rõ: 2% là phân vị khoảng cách cặp, tỉ lệ láng giềng hay 0.02? Thiếu quy ước cặp i<j, xử lý mật độ bằng nhau, điểm trùng và dc=0.

Ảnh hưởng: dc thay đổi mật độ, thứ tự rho*delta và tâm khởi tạo.

Cách xử lý: Xin quy tắc. Phân vị 2% khoảng cách cặp là một giả định có thể thử, chưa phải cấu hình đã công bố.

R12 - Một số công thức có dấu hiệu lỗi chỉ số

Vị trí: Trang 2, Eq. (4),(6),(7); trang 3, Eq. (12)

Thông tin trong bài: Eq. (7),(12) cộng theo i nhưng tâm vẫn mang k.

Thiếu / chưa rõ: Đọc đúng ký hiệu in thì mẫu số không đổi theo cụm đang cộng, dẫn membership=1/c khi khoảng cách khác 0. Eq. (6) có cận k=i; Eq. (4) ghi m cột thay vì c cụm; Eq. (10),(11) chưa thống nhất a/alpha.

Ảnh hưởng: Chép nguyên công thức có thể làm sai thuật toán. Chưa thể suy ra code tác giả cũng sai.

Cách xử lý: Sửa chỉ số cụm thành l ở mẫu số, membership có c cột; đối chiếu PFCM [10] và xin xác nhận.

R13 - Tiền xử lý và quy tắc số học chưa đầy đủ

Vị trí: Trang 3, mục 3.1 và Eq. (8)-(12)

Thông tin trong bài: Mỗi thuộc tính được MinMax về [0,1].

Thiếu / chưa rõ: Thiếu xử lý cột hằng, missing benchmark, ID/ngày tháng và kiểu dữ liệu. Khoảng cách 0, mẫu số 0 và lũy thừa lớn cần quy tắc epsilon, precision, cụm rỗng.

Ảnh hưởng: Có thể gây NaN/Inf hoặc thay đổi kết quả trên điểm trùng.

Cách xử lý: Lập schema dữ liệu và xử lý số học; công khai mọi lựa chọn thêm vào.

R14 - Tệp của 14 benchmark chưa được định danh

Vị trí: Trang 5, mục 4.1 và Table 1

Thông tin trong bài: Có tên, K, số thuộc tính và số bản ghi.

Thiếu / chưa rõ: Chỉ có trang chủ UCI; thiếu dataset ID, URL tệp, version, checksum, thứ tự dòng và schema. Flame,Jain,Compound,Aggregation,Room Occupancy cần xác nhận tệp cụ thể.

Ảnh hưởng: Cùng tên hoặc cùng N,F chưa đủ bảo đảm đúng dữ liệu/cột.

Cách xử lý: Tạo manifest nguồn/hash/features/labels; chỉ coi là tái hiện dữ liệu khi manifest khớp.

R15 - Glass K=7 cần giải thích

Vị trí: Trang 5, Table 1, Glass

Thông tin trong bài: Glass có K=7, F=9, N=214.

Thiếu / chưa rõ: UCI chính thức mô tả 6 lớp có dữ liệu; nhãn 4 vắng mặt. K=7 có thể là lựa chọn chủ động, nhưng bài không giải thích có dùng max(label) thay số lớp thực hay không.

Ảnh hưởng: 6 và 7 cụm cho kết quả khác nhau; không tự sửa K để làm đẹp metric.

Cách xử lý: Chế độ paper giữ k=7; thử k=6 riêng và xin xác nhận. Nguồn đối chiếu: UCI dataset 42.

R16 - Thiếu cấu hình các baseline

Vị trí: Trang 6, mục 4.3.1 và Table 5

Thông tin trong bài: So sánh K-means, FCM, PFCM, DP-FCM, DP-PFCM.

Thiếu / chưa rõ: Thiếu n_init/init/tol K-means; w/số vòng FCM; a,b,w,eta,K cho PFCM/DP-PFCM không GA. Table 5 không đủ thông số riêng SCA,FBI,A6,A7.

Ảnh hưởng: Tái tạo mô hình chính tốt vẫn có thể khác bảng vì baseline khác cấu hình.

Cách xử lý: Xin mọi config; dùng cùng preprocessing/seed/metric; ghi rõ giới hạn so sánh ngân sách.

R17 - Không có seed và kết quả thô 30 lần chạy

Vị trí: Trang 6, mục 4.3.1; trang 8, mục 4.3.2

Thông tin trong bài: Các số là trung bình của 30 lần thực thi.

Thiếu / chưa rõ: Thiếu seed, nhãn từng run, tâm, tham số tốt nhất, fitness trace và 30 giá trị gốc.

Ảnh hưởng: Không thể xác nhận khởi tạo, dao động và kiểm định đúng như tác giả.

Cách xử lý: Công bố 30 seed cố định và per-run CSV/tâm/nhãn/trace; xin raw results.

R18 - Cách tính Accuracy cần xác nhận implementation

Vị trí: Trang 5, Eq. (17)-(19)

Thông tin trong bài: Accuracy cộng max(a_i) theo cụm.

Thiếu / chưa rõ: Công thức nghiêng về purity nhưng corresponding true class chưa nêu mapping từng cụm hay một-một. NMI Eq. (19) đã rõ là geometric; thiếu code nên chưa xác minh implementation có đúng công thức.

Ảnh hưởng: Purity khác Hungarian accuracy; NMI arithmetic khác geometric.

Cách xử lý: Báo cáo purity theo cách hiểu Eq. (17); thêm Hungarian riêng, ARI chuẩn, NMI geometric. Xin code metric.

R19 - Table 15 chưa rõ cách tổng hợp SD

Vị trí: Trang 11, Table 15

Thông tin trong bài: Mỗi thuật toán có một SD cho mỗi metric.

Thiếu / chưa rõ: Có 14 dataset nhưng không ghi SD từng bộ: trung bình các SD, gộp 420 runs, hay SD của 14 means? Thiếu ddof; bảng không có hàng DP-FBI-PFCM.

Ảnh hưởng: Không tái tính được 0.012,0.010,0.059 từ bảng trung bình.

Cách xử lý: Xin raw runs và aggregation; báo cáo SD từng dataset trước.

R20 - Kiểm định thiếu input và cấu hình

Vị trí: Trang 6, Kruskal-Wallis; trang 8, Dunn-Bonferroni

Thông tin trong bài: Mức ý nghĩa 5%, Kruskal-Wallis và Dunn-Bonferroni.

Thiếu / chưa rõ: Thiếu raw runs, tên normality test, library/config, phạm vi family Bonferroni và cách xử lý seed ghép cặp. p-value làm tròn không đủ tái hiện.

Ảnh hưởng: Không tính lại đầy đủ Tables 9,10,14. Khác biệt thống kê tự nó không chỉ ra hướng tốt hơn.

Cách xử lý: Lưu per-run, đơn vị mẫu và family hiệu chỉnh; đối chiếu mean/effect size cùng p-value.

R21 - Glass Accuracy 0.066 mâu thuẫn 0.566

Vị trí: Trang 8, Table 11 A1; trang 6, Table 6 (5)

Thông tin trong bài: Cùng DP-PFCM: Table 6=0.566; Table 11=0.066.

Thiếu / chưa rõ: 0.066 còn thấp hơn cận 1/C của purity với các lớp Glass, không phù hợp cách hiểu Eq. (17). Có dấu hiệu lỗi in, chưa có đính chính.

Ảnh hưởng: Không dùng ô này làm mục tiêu khớp tuyệt đối hoặc tự sửa số liệu bài báo.

Cách xử lý: Giữ cả hai số với cờ conflict; xin xác nhận và kiểm tra lại hạng A1 nếu đính chính.

R22 - Xếp hạng khi hòa điểm chưa rõ

Vị trí: Trang 6, Table 6 Seed; trang 8, Table 11 Jain

Thông tin trong bài: Số in bằng nhau có thể có hạng khác hoặc cùng hạng.

Thiếu / chưa rõ: Seed FCM/DP-PFCM cùng 0.899 nhưng hạng 2/1; Jain ba số 0.881 cùng hạng 2. Thiếu full precision, tie rule và quy tắc phá hòa.

Ảnh hưởng: Không tái tính được sum-of-ranks chỉ từ số đã làm tròn.

Cách xử lý: Xin số chưa làm tròn và rule; xếp hạng full precision, công bố sensitivity.

R23 - Dẫn nguồn baseline A6,A7 bị đảo

Vị trí: Trang 8, mục 4.3.2; trang 13, [14],[28]

Thông tin trong bài: GA-IFWCOM dẫn [14]; SCA-FPCOM dẫn [28].

Thiếu / chưa rõ: Danh mục cho thấy [14] là SCA-FPCOM, [28] là GA-IFWCOM.

Ảnh hưởng: Có thể lấy nhầm bài nguồn khi viết lại baseline.

Cách xử lý: Đối chiếu tên bài/thuật toán thay số reference; xin config/code từng baseline.

R24 - Code, môi trường và đo runtime chưa đủ

Vị trí: Trang 5, đầu mục 4; trang 11, Table 16

Thông tin trong bài: Python, Windows 10, Intel Core i7, RAM 20 GB.

Thiếu / chưa rõ: PDF không có URL repo/config, Python/library versions, model CPU hoặc số luồng. Không rõ timer gồm DPC/preprocessing/tuning hay tổng 30 runs.

Ảnh hưởng: Không so được chính xác runtime/defaults. Không thấy link trong PDF không chứng minh code không tồn tại nơi khác.

Cách xử lý: Xin repo/commit/requirements/hardware/timer scope; công bố môi trường bản tái tạo.

R25 - Dữ liệu RFM và cleaning log chưa truy vết được

Vị trí: Trang 11, mục 5; trang 12, mục 5.1

Thông tin trong bài: Mendeley Data; 1288 khách, còn 1281 sau làm sạch.

Thiếu / chưa rõ: Thiếu DOI/URL/version, tên tệp, đơn vị Monetary, mốc Recency, ID 7 dòng bị loại. Điều kiện missing/âm áp dụng cho tất cả cột hay RFM?

Ảnh hưởng: Không bảo đảm cùng 1281 khách và số thành viên 6 cụm.

Cách xử lý: Xin DOI/checksum/cleaning log; chỉ R,F,M là features, ID giữ ngoài mô hình.

R26 - Elbow và output case study chưa đủ chi tiết

Vị trí: Trang 12, mục 5.1 và Table 18

Thông tin trong bài: Elbow chọn 6 cụm; bảng cho số khách và mức RFM.

Thiếu / chưa rõ: Không ghi thuật toán tạo Elbow, mục tiêu trục y, seed/n_init; thiếu tâm số, nhãn khách và a,b,w,eta cuối. Cụm 4/6 cùng Medium chưa đủ nhận diện lại.

Ảnh hưởng: Có thể dùng k=6, chưa tái hiện đường Elbow và Table 18.

Cách xử lý: Xin code Elbow và nhãn/tâm. Matching cụm qua tâm vì số hiệu cụm có thể hoán vị.

R27 - Abstract ghi 11 benchmark, thực nghiệm có 14

Vị trí: Trang 1, Abstract; trang 5, mục 4.1

Thông tin trong bài: Abstract ghi eleven; Table 1 có 14 bộ.

Thiếu / chưa rõ: Mâu thuẫn trình bày; ít nghiêm trọng hơn vấn đề fitness/GA.

Ảnh hưởng: Có thể chọn thiếu Room Occupancy,Rice,Wifi nếu chỉ theo Abstract.

Cách xử lý: Theo 14 bộ trong thực nghiệm; ghi discrepancy trong báo cáo.

Hồ sơ giả định cho bản tái tạo

Cần hai chế độ: bám sát số liệu công bố và kiểm tra các cách hiểu chưa rõ. Lựa chọn thêm vào phải có trong config, không quảng bá là cấu hình gốc.

Dữ liệu: Theo 14 bộ Table 1; lập manifest nguồn/hash/schema. Giữ Glass k=7 ở chế độ paper; thử k=6 riêng. Chưa có RFM đúng version thì case study chưa xác minh.

DPC: Gaussian density và rho*delta top-k đúng bài; dc 2% và tie là config. Nếu chọn phân vị khoảng cách, ghi là giả định.

PFCM: Sửa chỉ số nhất quán; a,b dương, w,eta>1. K, lịch gamma, epsilon và tol cần giả định/kiểm tra độ nhạy.

GA: Population 80, crossover 0.8, generations 100 theo Table 5. Mutation 0.01 và 0.1 chạy riêng. Fitness/bounds/precision/repair tách config.

Đánh giá: 30 seed cố định, lưu từng run. Purity theo cách hiểu Eq. (17), thêm Hungarian riêng, ARI chuẩn, NMI geometric. Công bố rule chuyển mu/t thành nhãn.

Đối chiếu: Giữ số bài thành tệp tham chiếu bất biến có cờ conflict. So mean/SD/effect size và runtime cùng máy; không chọn giả định chỉ để khớp bảng.

Thông tin nên xin tác giả

Repo/commit/requirements; file dữ liệu và preprocessing; fitness; bounds a,b,w,eta; K/lịch gamma; init/operators/repair/elitism; đính chính mutation rate và Glass 0.066; baseline config; 30 seed/raw scores; output RFM và code Elbow.

Nguồn và giới hạn đối chiếu

Nguồn chính: PDF gốc, DOI 10.1016/j.jestch.2023.101525. Không thấy link code trong PDF không chứng minh code chưa từng được công bố nơi khác.

UCI Glass: 214 mẫu, 9 features, 6 lớp có dữ liệu, nhãn 4 vắng mặt. Nguồn UCI chính thức.

scikit-learn NMI: hỗ trợ geometric/arithmetic; mặc định chuyển sang arithmetic từ 0.22. Tài liệu API chính thức.